Datové struktury

Grafy a hashování

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

20.10.2026

CC BY-NC-SA 4.0

Úvod

Od stromů ke grafům

Minule jsme prošli lineární struktury a stromy

Od stromů ke grafům

Minule jsme prošli lineární struktury a stromy

Co když ale vztahy mezi daty nejsou hierarchické?

Od stromů ke grafům

Minule jsme prošli lineární struktury a stromy

Co když ale vztahy mezi daty nejsou hierarchické? Mapa měst, sociální síť, počítačová síť…

Od stromů ke grafům

Minule jsme prošli lineární struktury a stromy

Co když ale vztahy mezi daty nejsou hierarchické? Mapa měst, sociální síť, počítačová síť…

Dnes nás čeká:

Od stromů ke grafům

Minule jsme prošli lineární struktury a stromy

Co když ale vztahy mezi daty nejsou hierarchické? Mapa měst, sociální síť, počítačová síť…

Dnes nás čeká:

- **Grafy** – reprezentace a procházení (BFS, DFS)

Od stromů ke grafům

Minule jsme prošli lineární struktury a stromy

Co když ale vztahy mezi daty nejsou hierarchické? Mapa měst, sociální síť, počítačová síť…

Dnes nás čeká:

- **Grafy** – reprezentace a procházení (BFS, DFS)
- **Hashovací tabulky** – vyhledávání v průměru za $\mathcal{O}(1)$

Grafové struktury

Graf (Graph)

Graf je

Graf (Graph)

Graf je datová struktura reprezentující

Graf (Graph)

Graf je datová struktura reprezentující **vztahy mezi objekty**

Graf (Graph)

Graf je datová struktura reprezentující **vztahy mezi objekty**

Skládá se z:

Graf (Graph)

Graf je datová struktura reprezentující **vztahy mezi objekty**

Skládá se z:

- **Vrcholů** (vertices/nodes) - objekty

Graf (Graph)

Graf je datová struktura reprezentující **vztahy mezi objekty**

Skládá se z:

- **Vrcholů** (vertices/nodes) - objekty
- **Hran** (edges) - spojení mezi objekty

Graf (Graph)

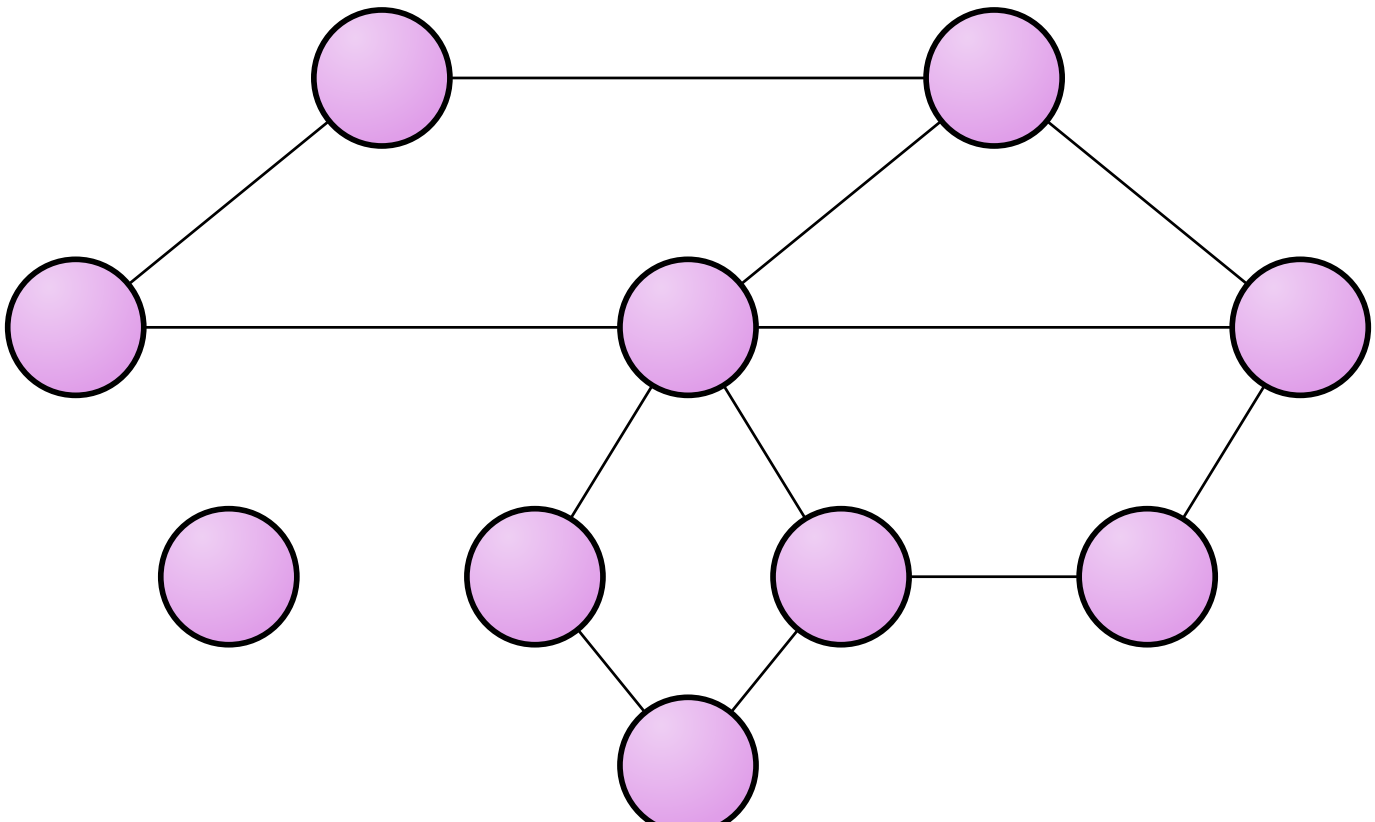
Graf je datová struktura reprezentující **vztahy mezi objekty**

Skládá se z:

- **Vrcholů** (vertices/nodes) - objekty
- **Hran** (edges) - spojení mezi objekty

Jedná se tedy o **zobecnění** datových struktur, které jsme již probírali (spojový seznam, stromy)

Vizualizace



Vizualizace

Graf je dvojice

Vizualizace

Graf je dvojice $G = (V, E)$, kde V je množina vrcholů a E je množina hran

Vizualizace

Graf je dvojice $G = (V, E)$, kde V je množina vrcholů a E je množina hran

Počet vrcholů označujeme jako $|V|$ a počet hran jako $|E|$

Příklady grafů v reálném světě

Příklady grafů v reálném světě

- **Sociální sítě**

Příklady grafů v reálném světě

- **Sociální sítě** - lidé (vrcholy), přátelství (hrany)

Příklady grafů v reálném světě

- **Sociální sítě** - lidé (vrcholy), přátelství (hrany)
- **Mapy**

Příklady grafů v reálném světě

- **Sociální sítě** - lidé (vrcholy), přátelství (hrany)
- **Mapy** - města (vrcholy), silnice (hrany)

Příklady grafů v reálném světě

- **Sociální sítě** - lidé (vrcholy), přátelství (hrany)
- **Mapy** - města (vrcholy), silnice (hrany)
- **Internet**

Příklady grafů v reálném světě

- **Sociální sítě** - lidé (vrcholy), přátelství (hrany)
- **Mapy** - města (vrcholy), silnice (hrany)
- **Internet** - servery (vrcholy), spojení (hrany)

Příklady grafů v reálném světě

- **Sociální sítě** - lidé (vrcholy), přátelství (hrany)
- **Mapy** - města (vrcholy), silnice (hrany)
- **Internet** - servery (vrcholy), spojení (hrany)
- **Elektrické obvody**

Příklady grafů v reálném světě

- **Sociální sítě** - lidé (vrcholy), přátelství (hrany)
- **Mapy** - města (vrcholy), silnice (hrany)
- **Internet** - servery (vrcholy), spojení (hrany)
- **Elektrické obvody** - komponenty (vrcholy), dráty (hrany)

Neorientovaný graf

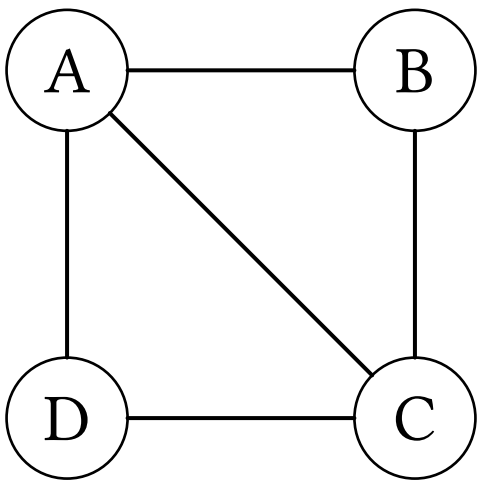
Neorientovaný graf (Undirected) - hrany

Neorientovaný graf

Neorientovaný graf (Undirected) - hrany **nemají** směr

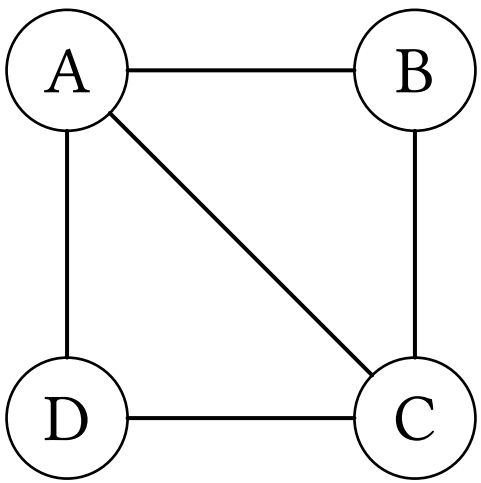
Neorientovaný graf

Neorientovaný graf (Undirected) - hrany **nemají** směr



Neorientovaný graf

Neorientovaný graf (Undirected) - hrany **nemají** směr



$$E = \{\{A, B\}, \{B, C\}, \{C, D\}, \{D, A\}, \{A, C\}\}$$

Orientovaný graf

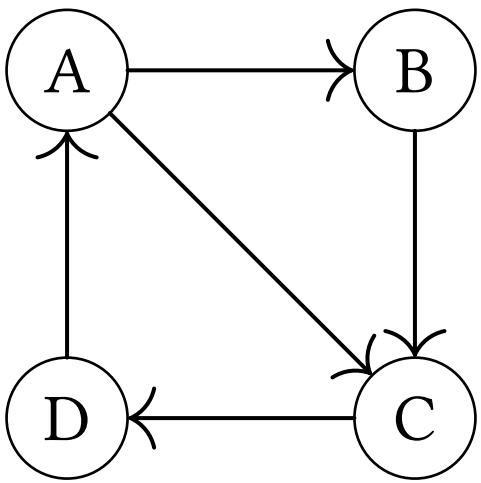
Orientovaný graf (Directed) - hrany

Orientovaný graf

Orientovaný graf (Directed) - hrany **mají** směr

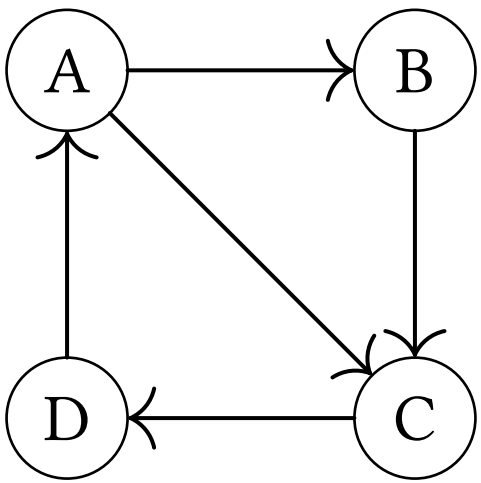
Orientovaný graf

Orientovaný graf (Directed) - hrany **mají** směr



Orientovaný graf

Orientovaný graf (Directed) - hrany **mají** směr



$$E = \{(A, B), (B, C), (C, D), (D, A), (A, C)\}$$

Orientovaný graf

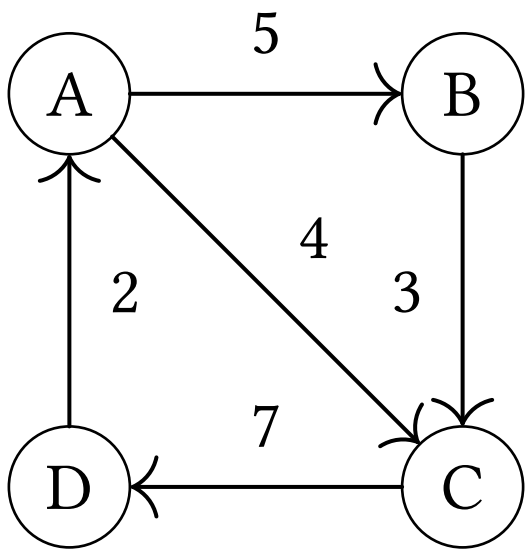
Vážený graf (Weighted) - hrany mají

Orientovaný graf

Vážený graf (Weighted) - hrany mají **váhu** (vzdálenost, cena, ...)

Orientovaný graf

Vážený graf (Weighted) - hrany mají **váhu** (vzdálenost, cena, ...)



Reprezentace grafů

Matice sousednosti

Matice sousednosti

2D pole reprezentující hrany mezi vrcholy

```
// Graph: A-B, A-C, B-C
const graph: number[][] = [
//   A  B  C
    [0, 1, 1], // A
    [1, 0, 1], // B
    [1, 1, 0]  // C
];
```

Matice sousednosti - složitost operací

Prostorová složitost:

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Časová složitost kontroly existence hrany:

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Časová složitost kontroly existence hrany: $\mathcal{O}(1)$

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Časová složitost kontroly existence hrany: $\mathcal{O}(1)$

Časová složitost přidání hrany:

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Časová složitost kontroly existence hrany: $\mathcal{O}(1)$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Časová složitost kontroly existence hrany: $\mathcal{O}(1)$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany:

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Časová složitost kontroly existence hrany: $\mathcal{O}(1)$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany: $\mathcal{O}(1)$

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Časová složitost kontroly existence hrany: $\mathcal{O}(1)$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany: $\mathcal{O}(1)$

Časová složitost přidání vrcholu:

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Časová složitost kontroly existence hrany: $\mathcal{O}(1)$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany: $\mathcal{O}(1)$

Časová složitost přidání vrcholu: $\mathcal{O}(|V|^2)$

Matice sousednosti - složitost operací

Prostorová složitost: $\mathcal{O}(|V|^2)$

Časová složitost kontroly existence hrany: $\mathcal{O}(1)$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany: $\mathcal{O}(1)$

Časová složitost přidání vrcholu: $\mathcal{O}(|V|^2)$

- Přidání vrcholu vyžaduje realokaci každého řádku v 2D poli

Matice sousednosti - složitost operací

Vhodné pro husté grafy (mnoho hran)

Seznam sousedů

Seznam sousedů

Pro každý vrchol ukládáme (spojový) seznam jeho sousedů

Seznam sousedů

Pro každý vrchol ukládáme (spojový) seznam jeho sousedů

- V praxi je typické použití i dynamické pole

```
type Graph = Record<string, string[]>;

const graph: Graph = {
  'A': ['B', 'C'],
  'B': ['A', 'C', 'D'],
  'C': ['A', 'B'],
  'D': ['B']
};
```

Seznam sousedů - složitost operací

Pomocí $\Delta(v)$ označujeme stupeň vrcholu v (počet sousedů)

Seznam sousedů - složitost operací

Pomocí $\Delta(v)$ označujeme stupeň vrcholu v (počet sousedů)

Pomocí $\Delta(G)$ označujeme maximální stupeň vrcholu v grafu G

Seznam sousedů - složitost operací

Prostorová složitost:

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|)$

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je $\mathcal{O}(|V| * \Delta(G))$

Časová složitost kontroly existence hrany:

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je $\mathcal{O}(|V| * \Delta(G))$

Časová složitost kontroly existence hrany: $\mathcal{O}(\Delta(G))$

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je $\mathcal{O}(|V| * \Delta(G))$

Časová složitost kontroly existence hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání hrany:

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je $\mathcal{O}(|V| * \Delta(G))$

Časová složitost kontroly existence hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je $\mathcal{O}(|V| * \Delta(G))$

Časová složitost kontroly existence hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany:

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je $\mathcal{O}(|V| * \Delta(G))$

Časová složitost kontroly existence hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany: $\mathcal{O}(\Delta(G))$

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je $\mathcal{O}(|V| * \Delta(G))$

Časová složitost kontroly existence hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání vrcholu:

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je $\mathcal{O}(|V| * \Delta(G))$

Časová složitost kontroly existence hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání vrcholu: $\mathcal{O}(|V|)$

Seznam sousedů - složitost operací

Prostorová složitost: $\mathcal{O}(|V| + |E|) = \mathcal{O}(|V|^2)$

- Alternativní odhad je $\mathcal{O}(|V| * \Delta(G))$

Časová složitost kontroly existence hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání hrany: $\mathcal{O}(1)$

Časová složitost odebrání hrany: $\mathcal{O}(\Delta(G))$

Časová složitost přidání vrcholu: $\mathcal{O}(|V|)$

- Přidání vrcholu vyžaduje realokaci hlavního seznamu sousedů

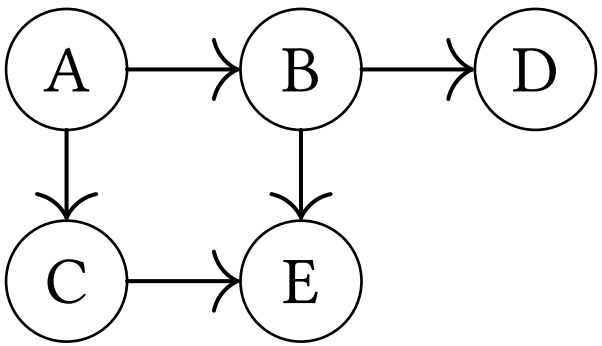
Vhodnost použití

Vhodné pro řídké grafy (málo hran)

Speciální typy grafů

Acyklický graf (DAG - Directed Acyclic Graph)

Acyklický graf (DAG - Directed Acyclic Graph)



Úplný graf

Úplný graf je takový graf, ve kterém

Úplný graf

Úplný graf je takový graf, ve kterém je každý vrchol spojen se všemi ostatními a značíme ho K_n

Úplný graf

Úplný graf je takový graf, ve kterém je každý vrchol spojen se všemi ostatními a značíme ho K_n



K_1

Úplný graf

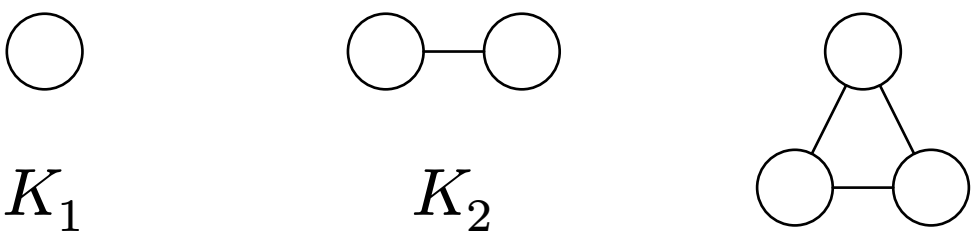
Úplný graf je takový graf, ve kterém je každý vrchol spojen se všemi ostatními a značíme ho K_n



K_1 K_2

Úplný graf

Úplný graf je takový graf, ve kterém je každý vrchol spojen se všemi ostatními a značíme ho K_n



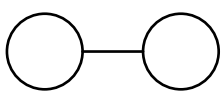
K_1 K_2 K_3

Úplný graf

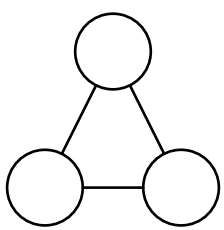
Úplný graf je takový graf, ve kterém je každý vrchol spojen se všemi ostatními a značíme ho K_n



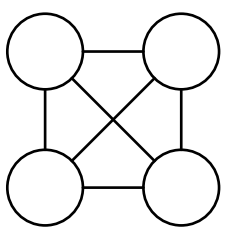
K_1



K_2



K_3



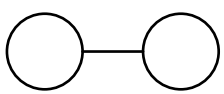
K_4

Úplný graf

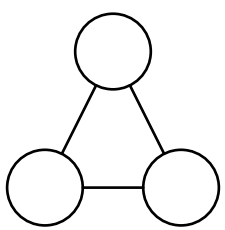
Úplný graf je takový graf, ve kterém je každý vrchol spojen se všemi ostatními a značíme ho K_n



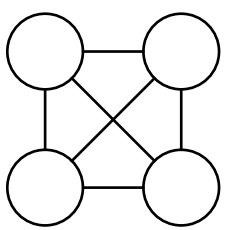
K_1



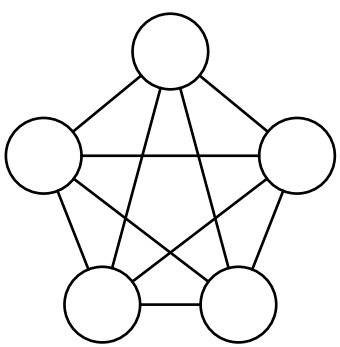
K_2



K_3



K_4



K_5

Bipartitní graf

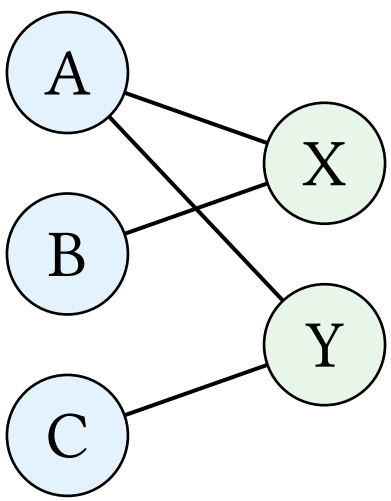
Bipartitní graf je takový graf, ve kterém

Bipartitní graf

Bipartitní graf je takový graf, ve kterém jsou vrcholy rozděleny do dvou množin a hrany vedou pouze mezi vrcholy z různých množin

Bipartitní graf

Bipartitní graf je takový graf, ve kterém jsou vrcholy rozděleny do dvou množin a hrany vedou pouze mezi vrcholy z různých množin



Strom

Strom je

Strom

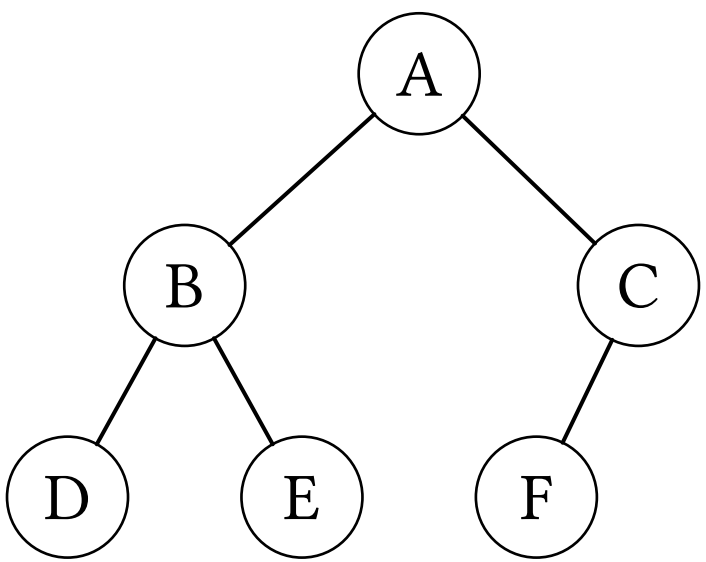
Strom je souvislý acyklický graf

- Z každého vrcholu existuje cesta do jakéhokoli jiného vrcholu
- Neobsahuje cykly

Strom

Strom je souvislý acyklický graf

- Z každého vrcholu existuje cesta do jakéhokoli jiného vrcholu
- Neobsahuje cykly



Procházení grafů

Procházení grafů

Dva hlavní způsoby procházení grafů:

Procházení grafů

Dva hlavní způsoby procházení grafů:

1. **BFS (Breadth-First Search)** - do šířky

Procházení grafů

Dva hlavní způsoby procházení grafů:

1. **BFS (Breadth-First Search)** - do šířky
2. **DFS (Depth-First Search)** - do hloubky

BFS (Breadth-First Search)

BFS - Prohledávání do šířky

BFS (Breadth-First Search)

BFS - Prohledávání do šířky

Procházení po úrovních (vrstvách)

BFS (Breadth-First Search)

BFS - Prohledávání do šířky

Procházení po úrovních (vrstvách)

Využívá **frontu** (FIFO)

BFS (Breadth-First Search)

BFS - Prohledávání do šířky

Procházení po úrovních (vrstvách)

Využívá **frontu** (FIFO)

Aplikace:

BFS (Breadth-First Search)

BFS - Prohledávání do šířky

Procházení po úrovních (vrstvách)

Využívá **frontu** (FIFO)

Aplikace:

- Hledání nejkratší cesty v nevážených grafech

BFS (Breadth-First Search)

BFS - Prohledávání do šířky

Procházení po úrovních (vrstvách)

Využívá **frontu** (FIFO)

Aplikace:

- Hledání nejkratší cesty v nevážených grafech
- Hledání komponent souvislosti

BFS (Breadth-First Search)

BFS - Prohledávání do šířky

Procházení po úrovních (vrstvách)

Využívá **frontu** (FIFO)

Aplikace:

- Hledání nejkratší cesty v nevážených grafech
- Hledání komponent souvislosti
- Web scraping

BFS (Breadth-First Search)

BFS - Prohledávání do šířky

Procházení po úrovních (vrstvách)

Využívá **frontu** (FIFO)

Aplikace:

- Hledání nejkratší cesty v nevážených grafech
- Hledání komponent souvislosti
- Web scraping
- Procházení stavového prostoru

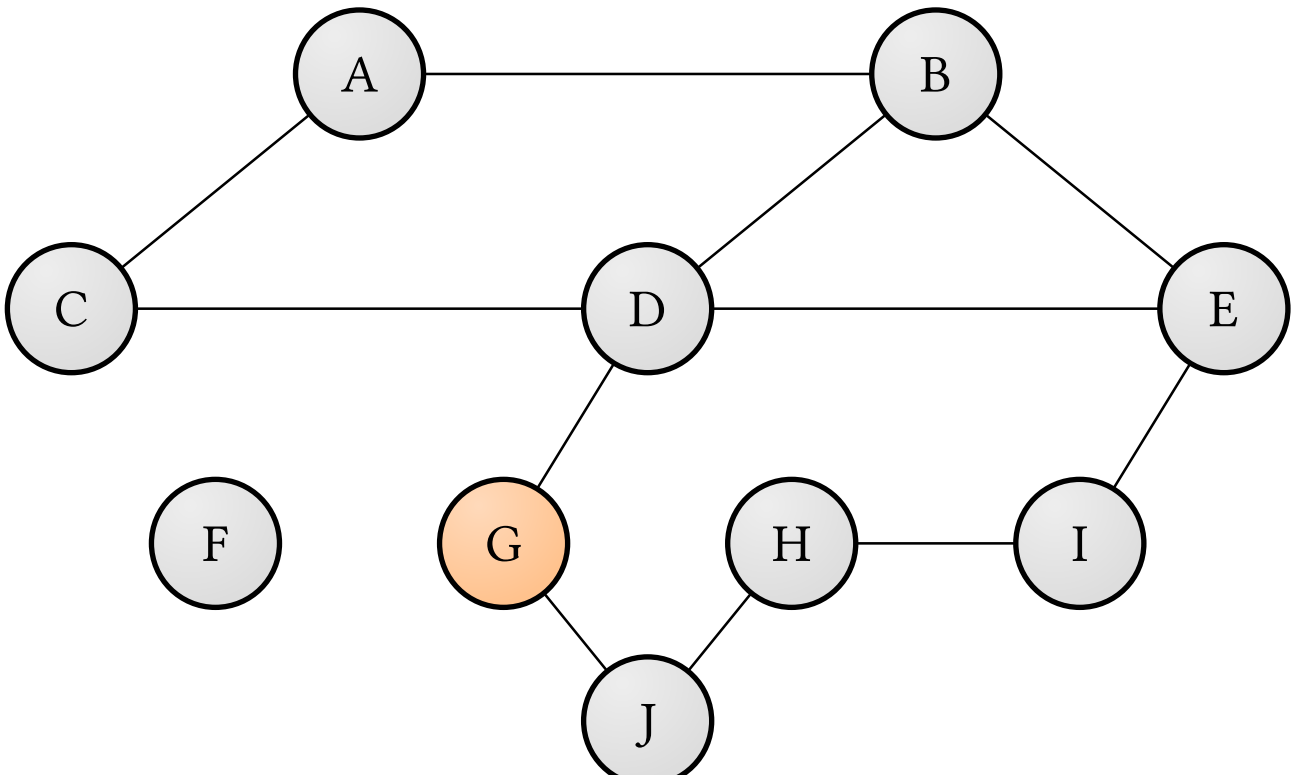
Příklad BFS

Aktuální: G

Fronta:
[G]

Legenda:

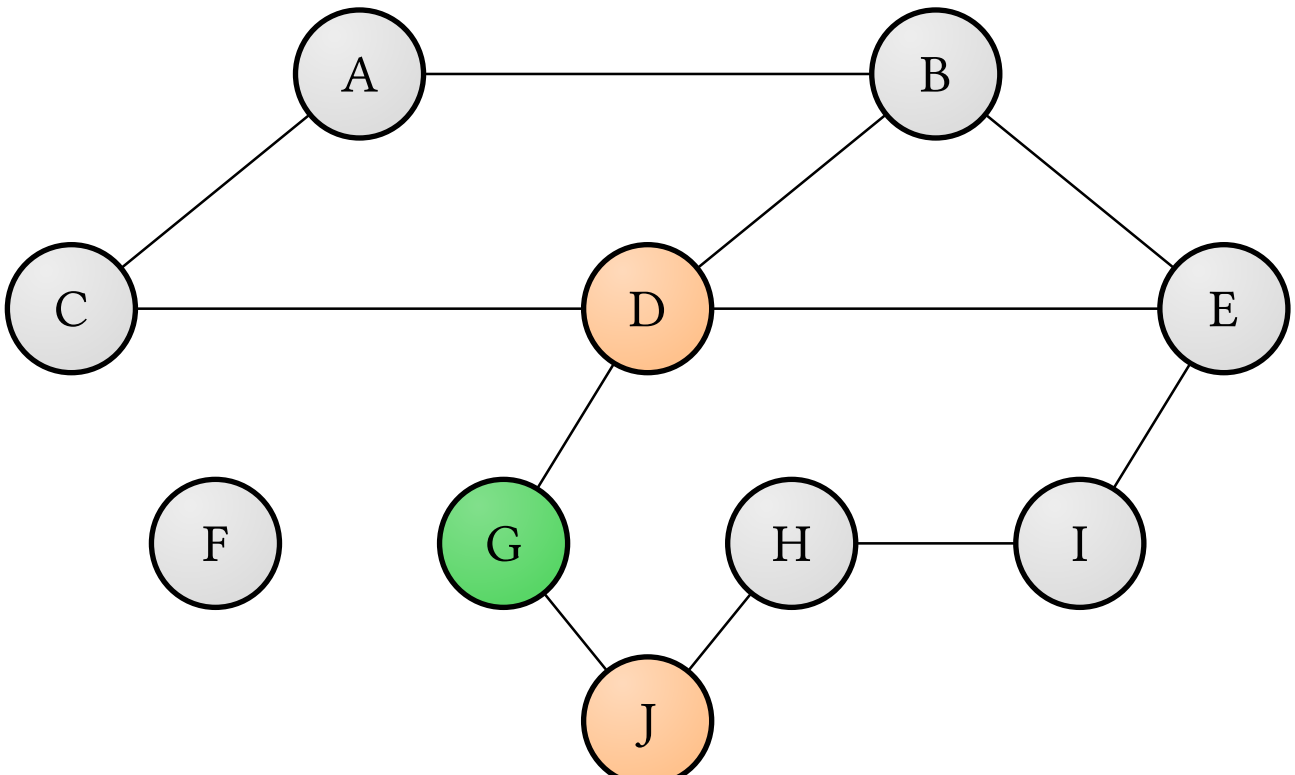
- Nenavštívené
- Ve frontě
- Zpracované



Příklad BFS

Aktuální: D

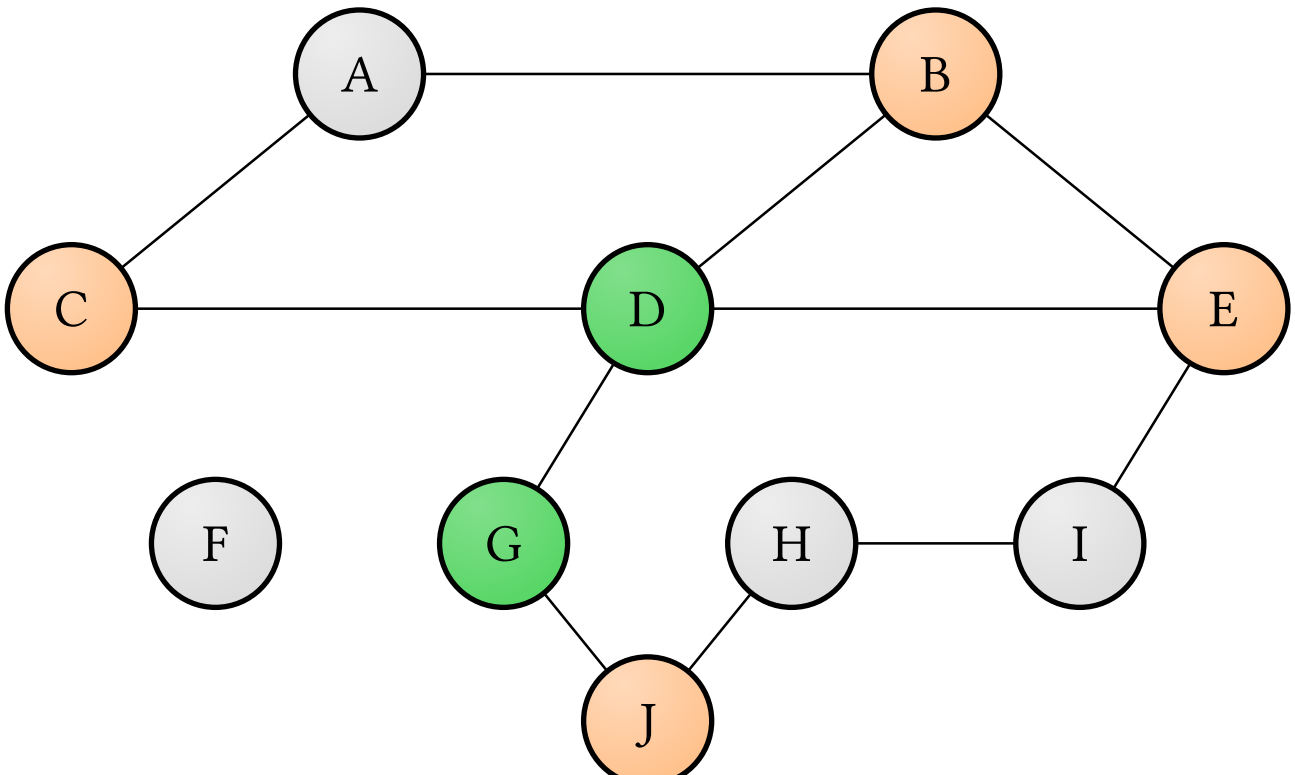
Fronta:
[D, J]



Příklad BFS

Aktuální: J

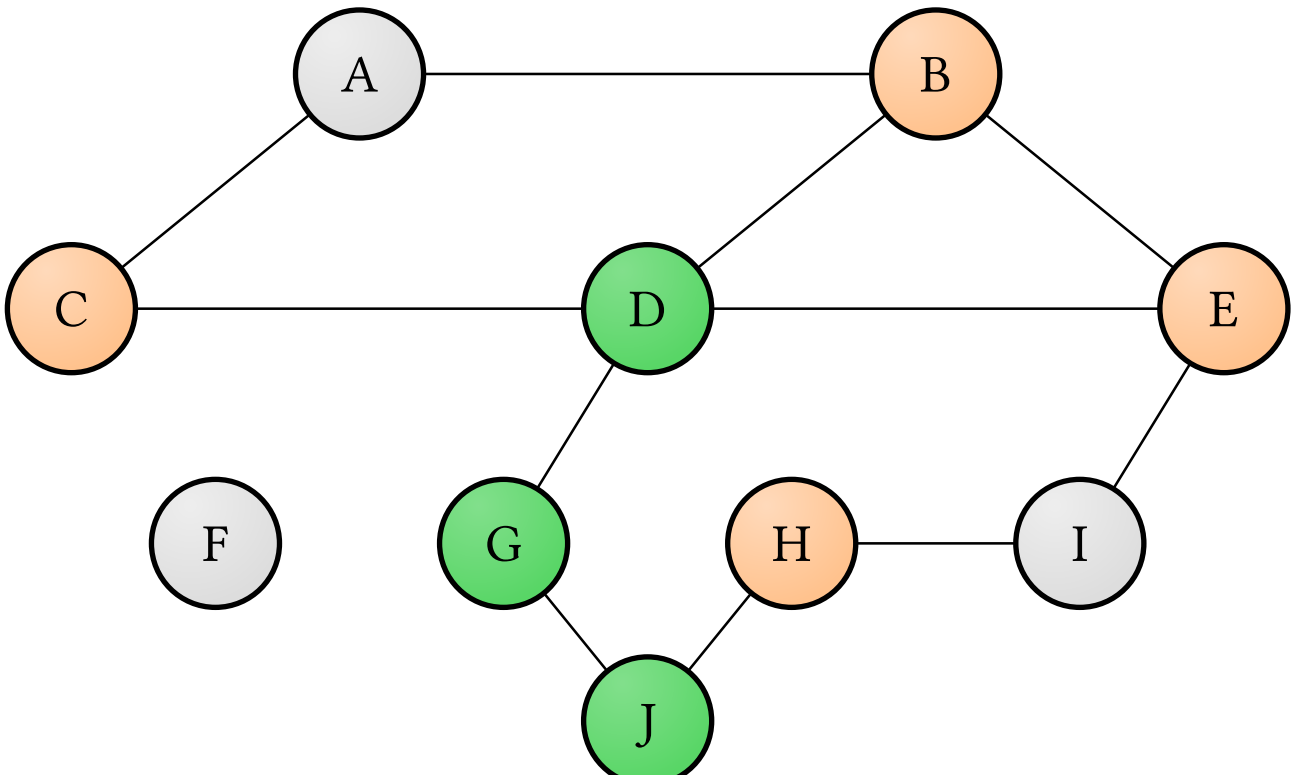
Fronta:
[J, B, C, E]



Příklad BFS

Aktuální: B

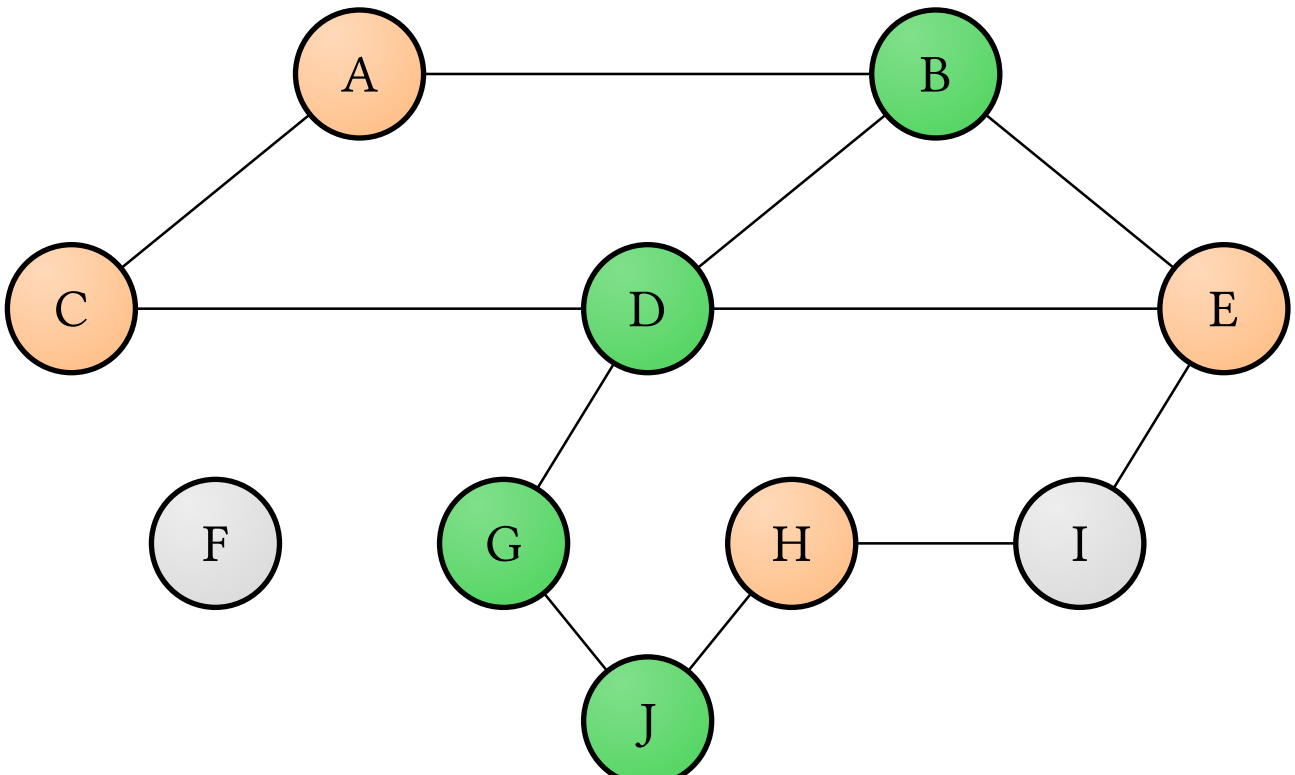
Fronta:
[B, C, E, H]



Příklad BFS

Aktuální: C

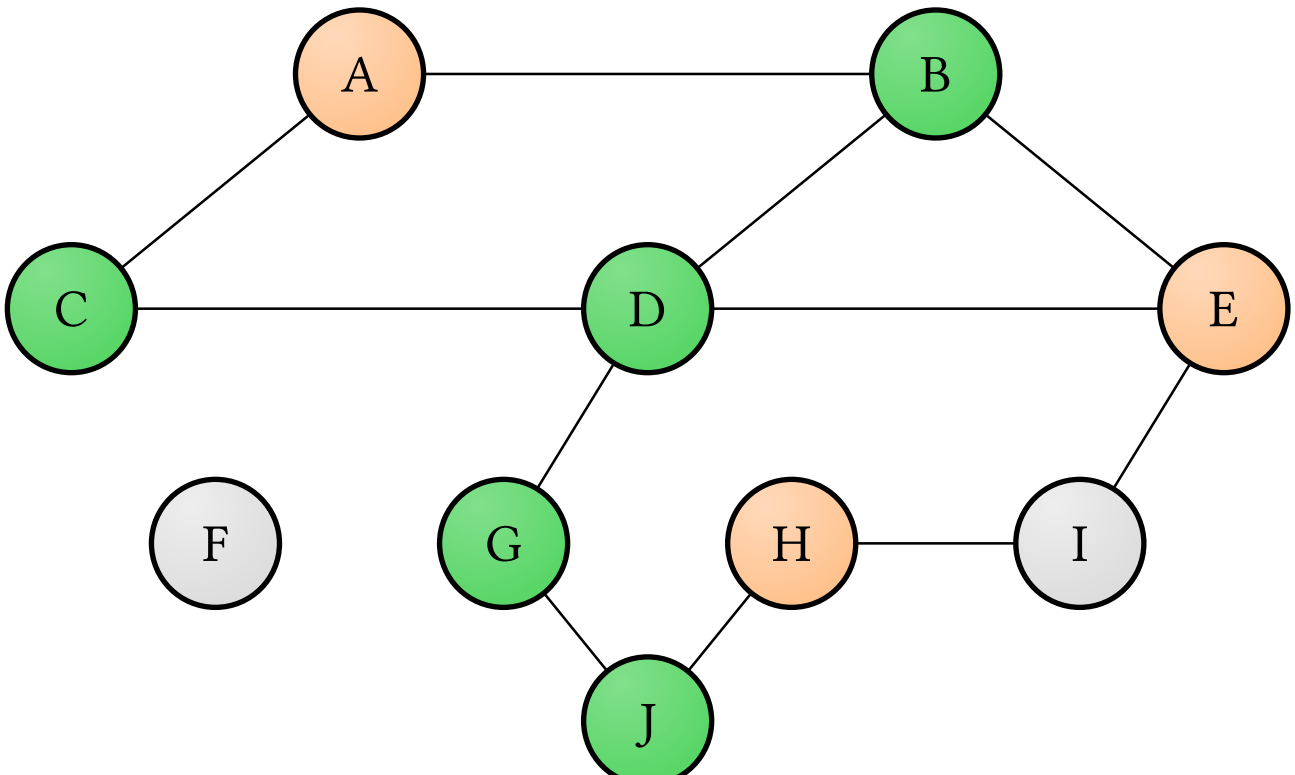
Fronta:
[C, E, H, A]



Příklad BFS

Aktuální: E

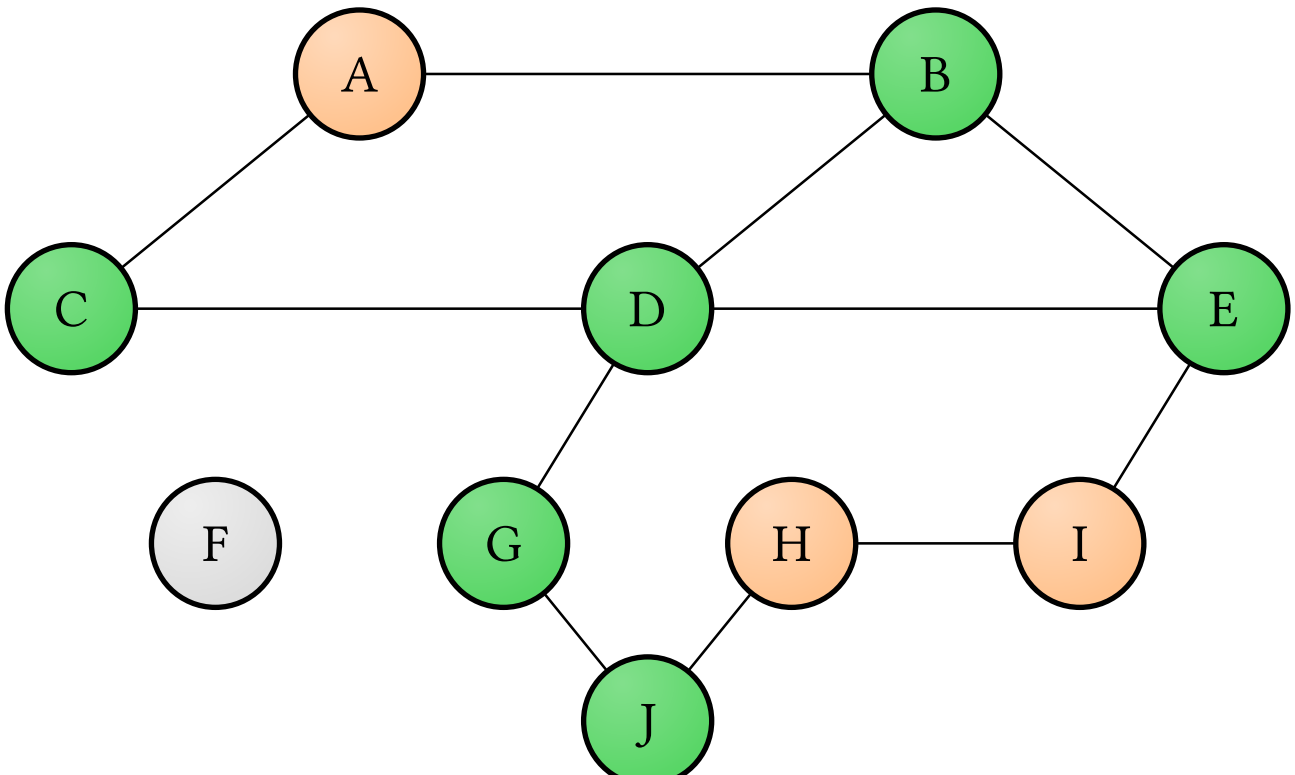
Fronta:
[E, H, A]



Příklad BFS

Aktuální: H

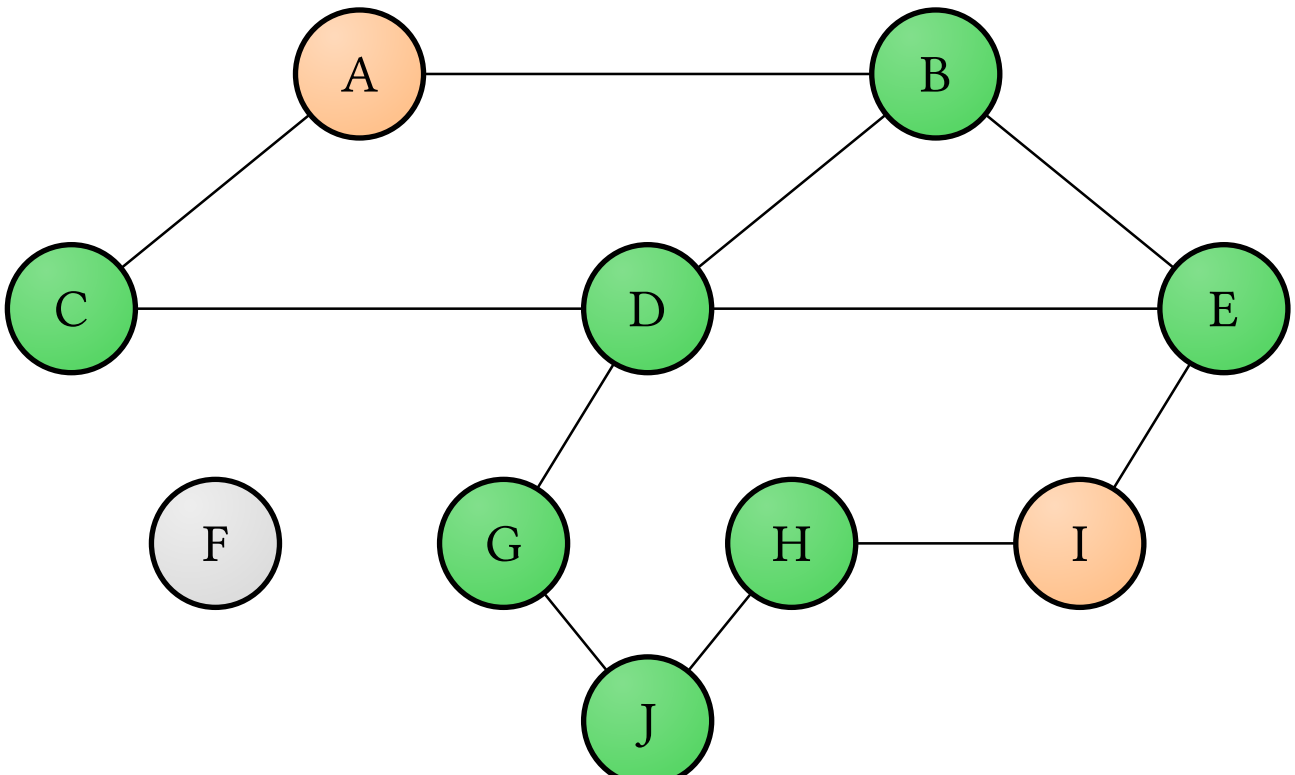
Fronta:
[H, A, I]



Příklad BFS

Aktuální: A

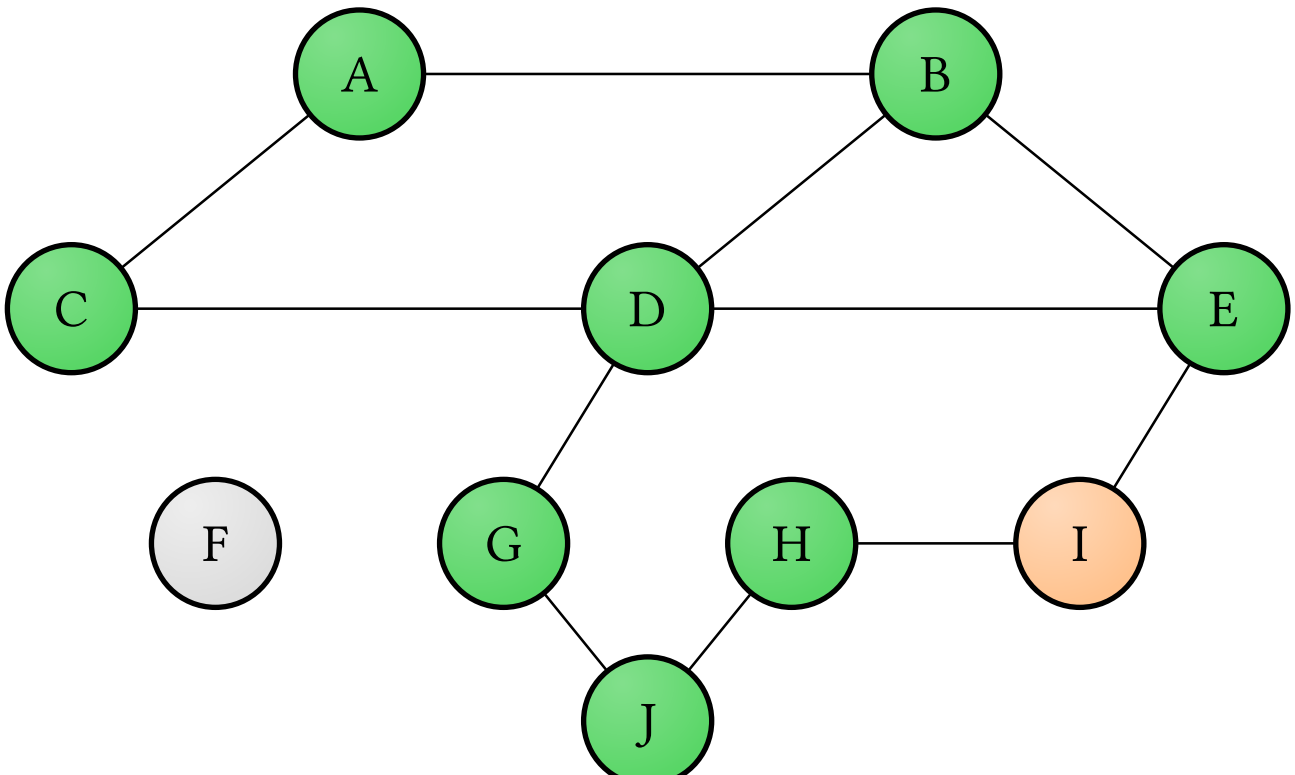
Fronta:
[A, I]



Příklad BFS

Aktuální: I

Fronta:
[I]



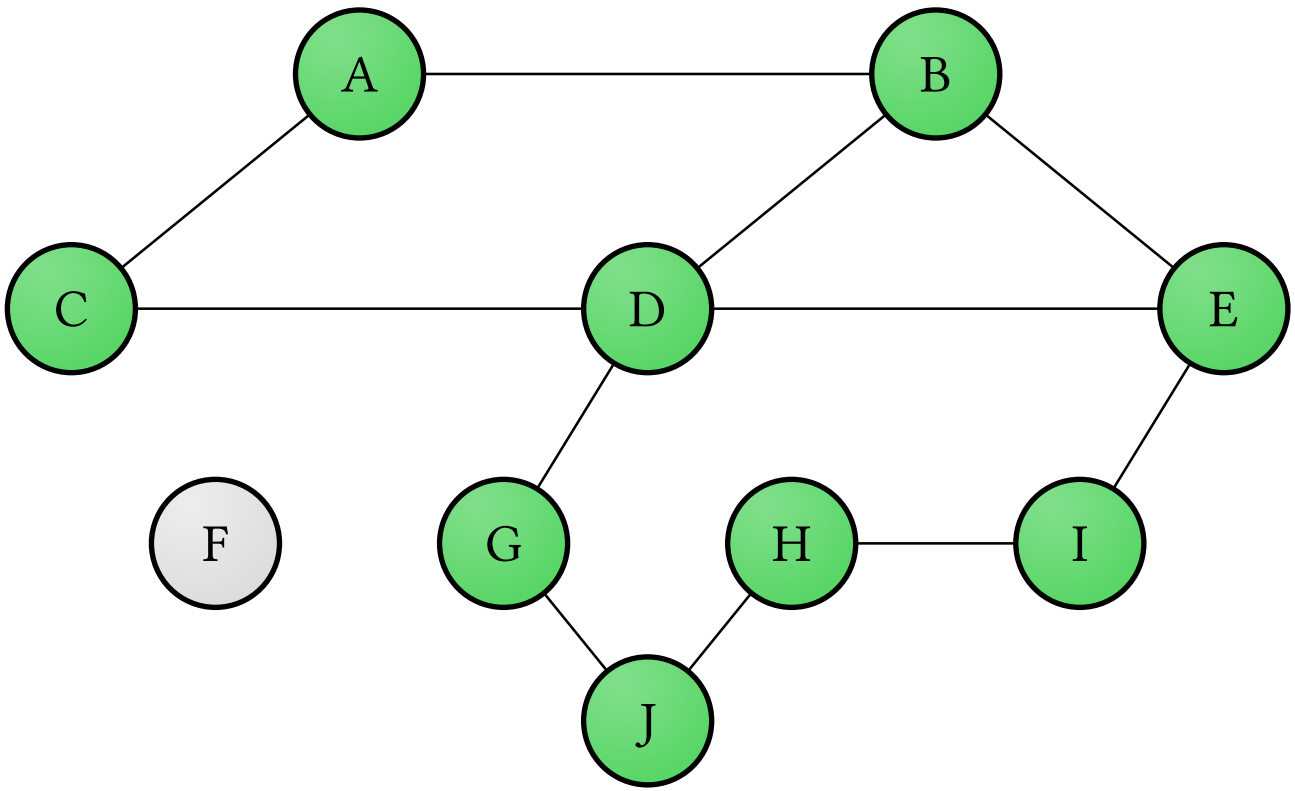
Příklad BFS

Aktuální: -

Fronta:
[ ]

BFS dokončeno!

Vrchol F není dosažitelný
z vrcholu G.



Implementace BFS

```
function bfs(graph: Graph, start: string): void {
  const visited = new Set<string>();
  const queue = new Queue<string>();

  queue.enqueue(start);
  visited.add(start);
```

Implementace BFS

```
  while (!queue.isEmpty()) {
    const vertex = queue.dequeue()!;
    console.log(vertex);
    for (const neighbor of graph[vertex]) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.enqueue(neighbor);
      }
    }
  }
}
```

DFS (Depth-First Search)

DFS - Prohledávání do hloubky

DFS (Depth-First Search)

DFS - Prohledávání do hloubky

Procházení co nejdále do hloubky, pak zpět

DFS (Depth-First Search)

DFS - Prohledávání do hloubky

Procházení co nejdále do hloubky, pak zpět

Využívá **zásobník** (LIFO) nebo rekurzi

DFS (Depth-First Search)

DFS - Prohledávání do hloubky

Procházení co nejdále do hloubky, pak zpět

Využívá **zásobník** (LIFO) nebo rekurzi

Aplikace:

DFS (Depth-First Search)

DFS - Prohledávání do hloubky

Procházení co nejdále do hloubky, pak zpět

Využívá **zásobník** (LIFO) nebo rekurzi

Aplikace:

- Detekce cyklů

DFS (Depth-First Search)

DFS - Prohledávání do hloubky

Procházení co nejdále do hloubky, pak zpět

Využívá **zásobník** (LIFO) nebo rekurzi

Aplikace:

- Detekce cyklů
- Topologické třídění

DFS (Depth-First Search)

DFS - Prohledávání do hloubky

Procházení co nejdále do hloubky, pak zpět

Využívá **zásobník** (LIFO) nebo rekurzi

Aplikace:

- Detekce cyklů
- Topologické třídění
- Hledání silně souvislých komponent

DFS (Depth-First Search)

DFS - Prohledávání do hloubky

Procházení co nejdále do hloubky, pak zpět

Využívá **zásobník** (LIFO) nebo rekurzi

Aplikace:

- Detekce cyklů
- Topologické třídění
- Hledání silně souvislých komponent
- Řešení bludišť a puzzle

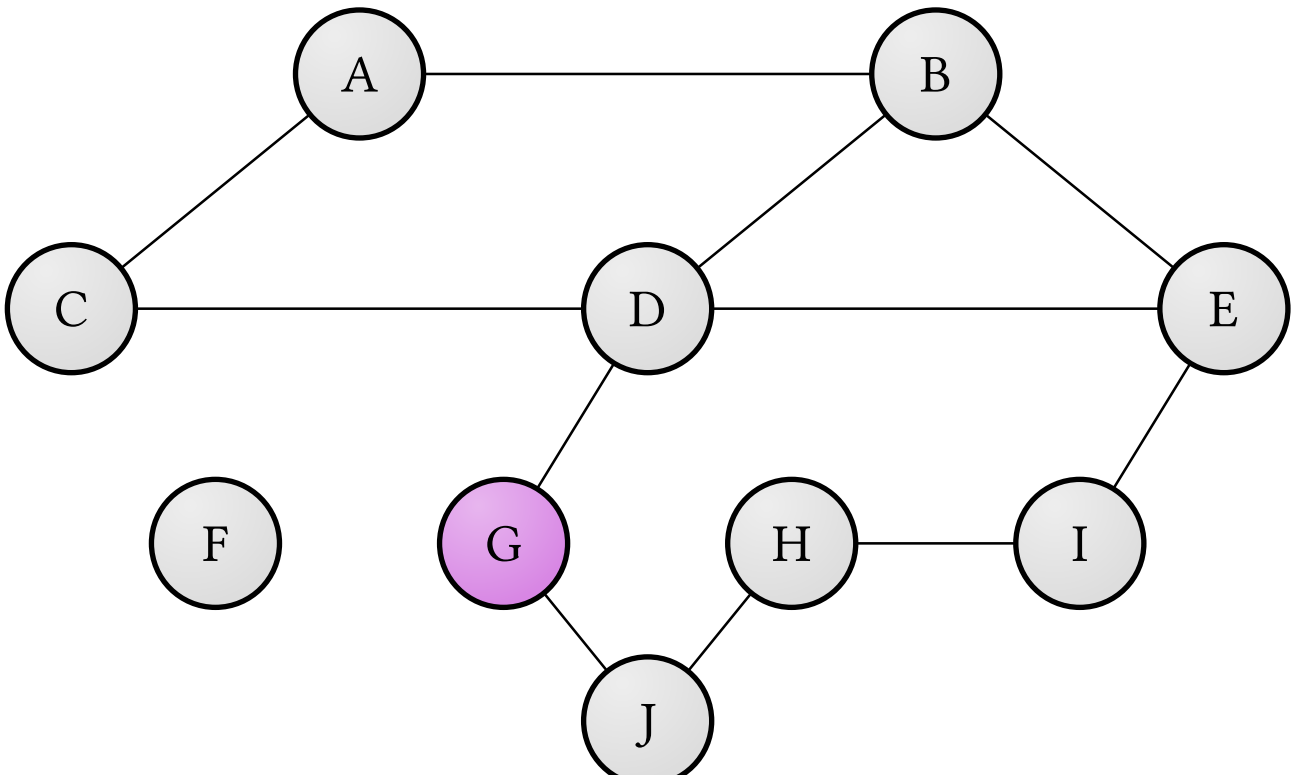
Příklad DFS

Aktuální: G

Zásobník:
[G]

Legenda:

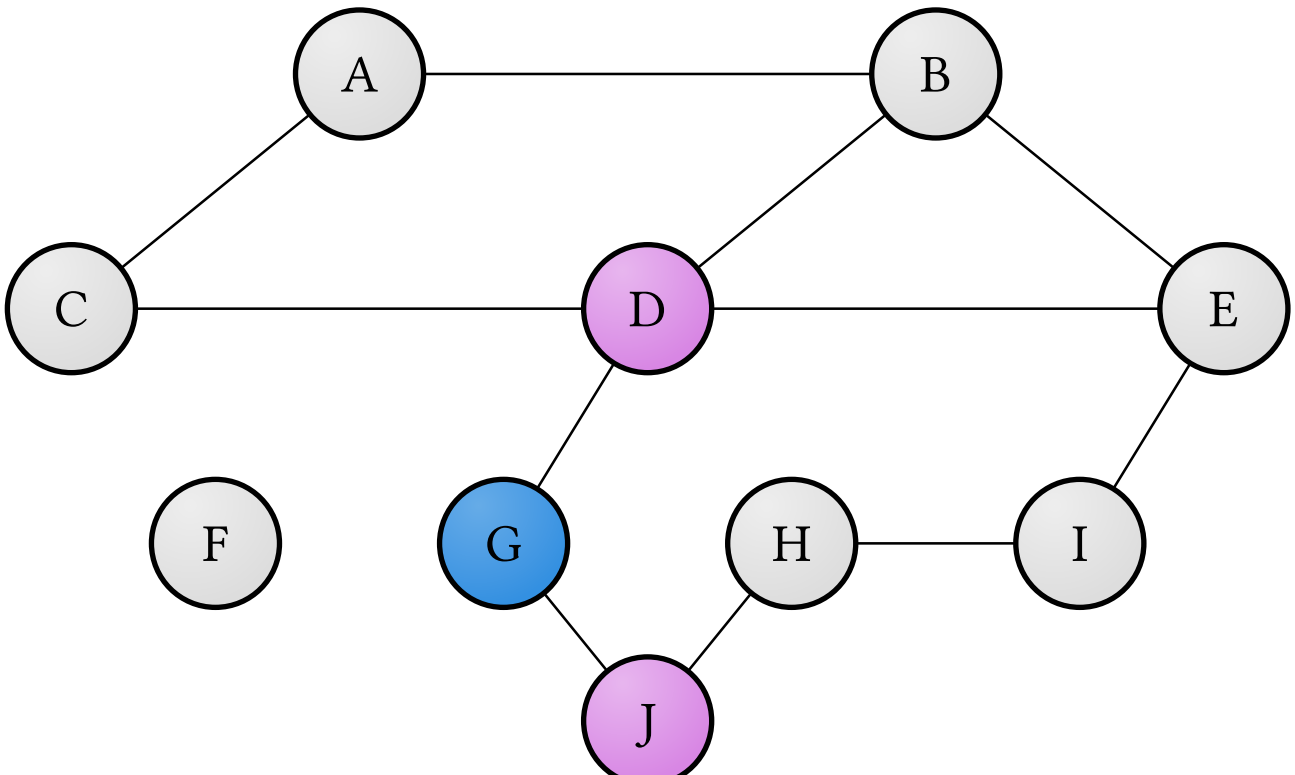
- Nenavštívené
- Na zásobníku
- Zpracované



Příklad DFS

Aktuální: J

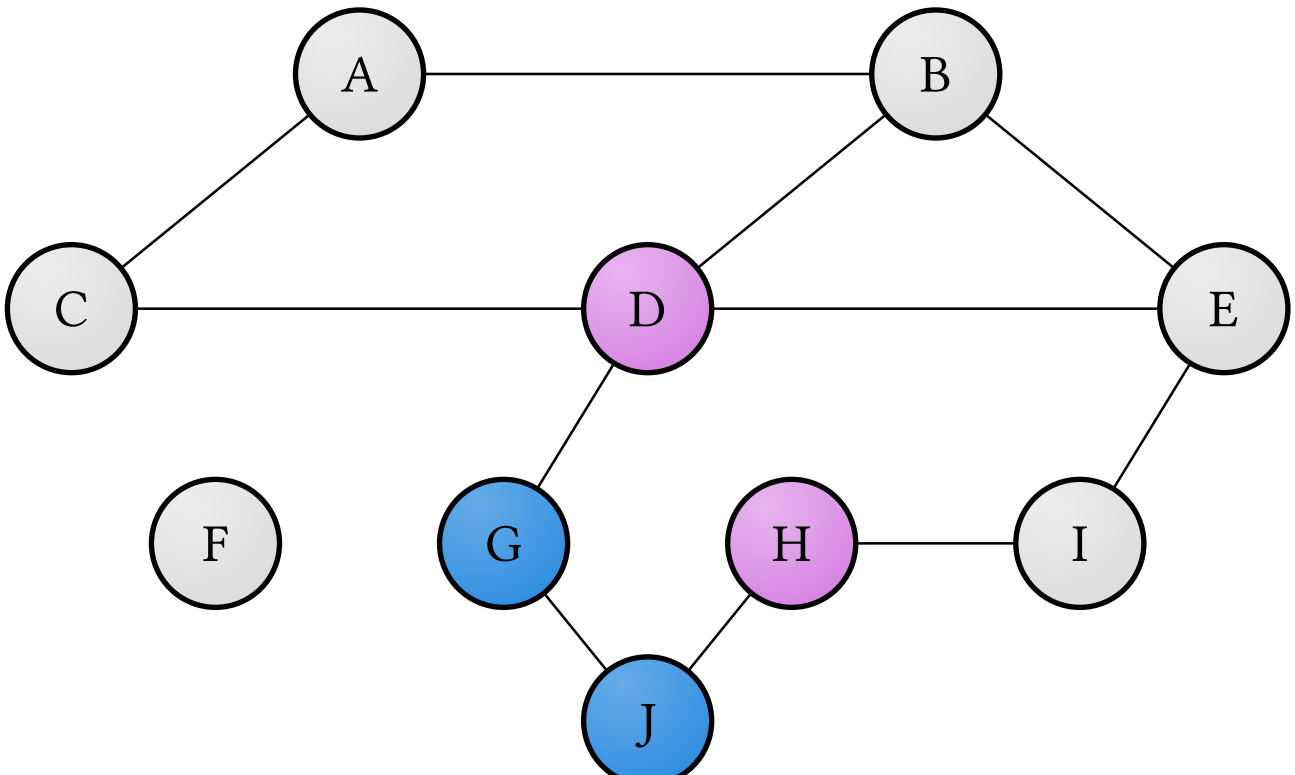
Zásobník:
[D, J]



Příklad DFS

Aktuální: H

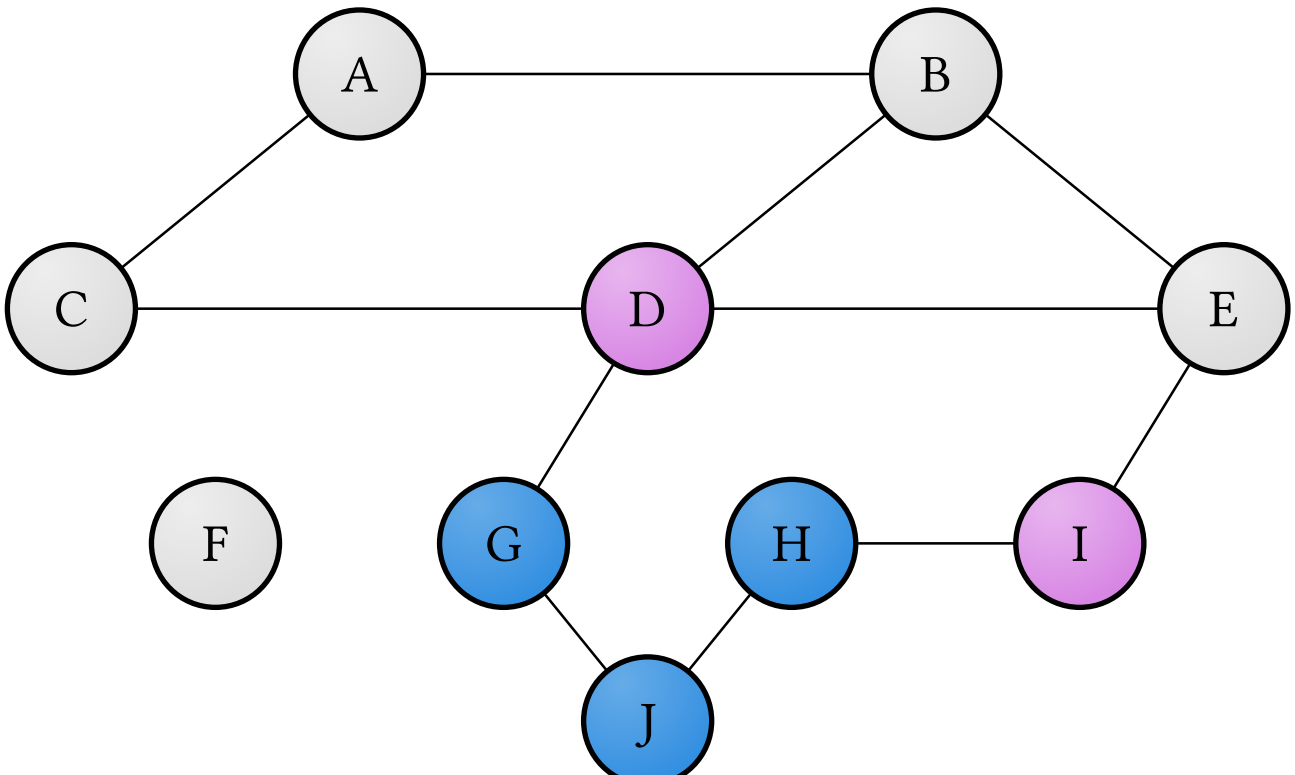
Zásobník:
[D, H]



Příklad DFS

Aktuální: I

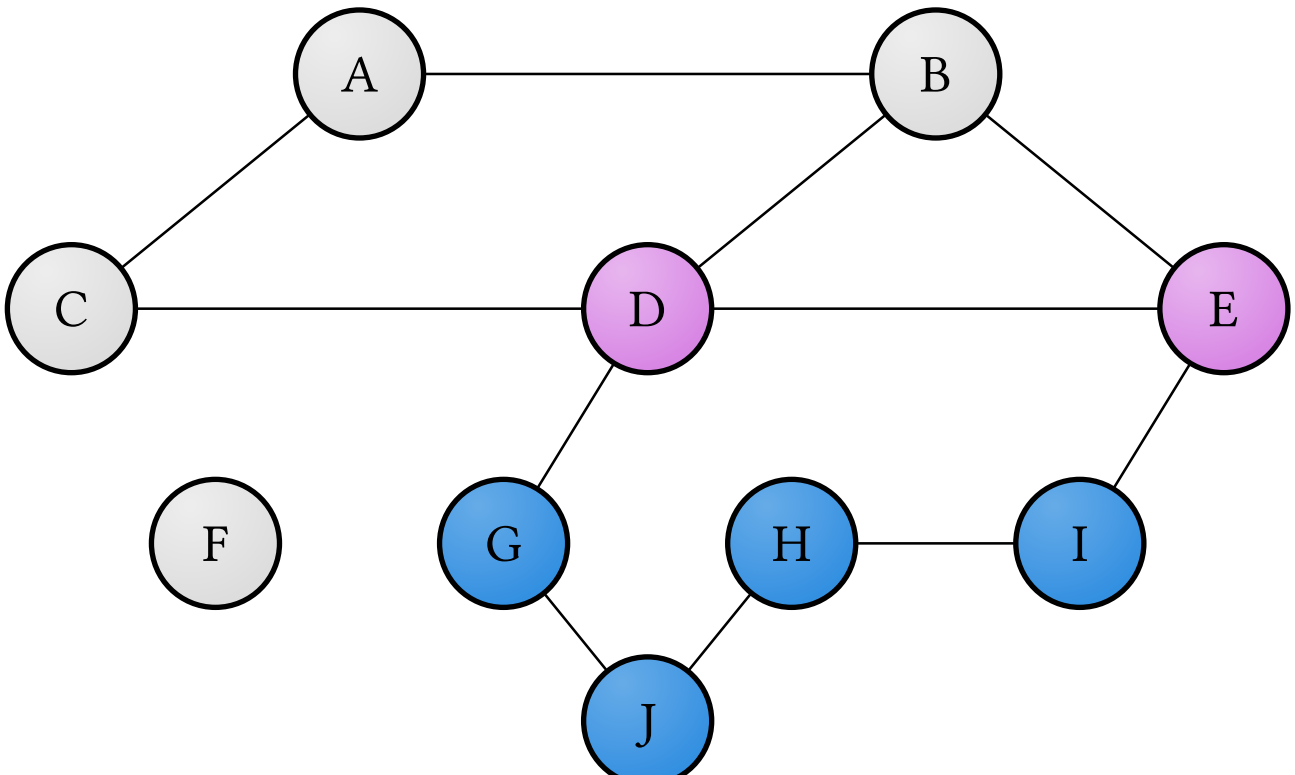
Zásobník:
[D, I]



Příklad DFS

Aktuální: E

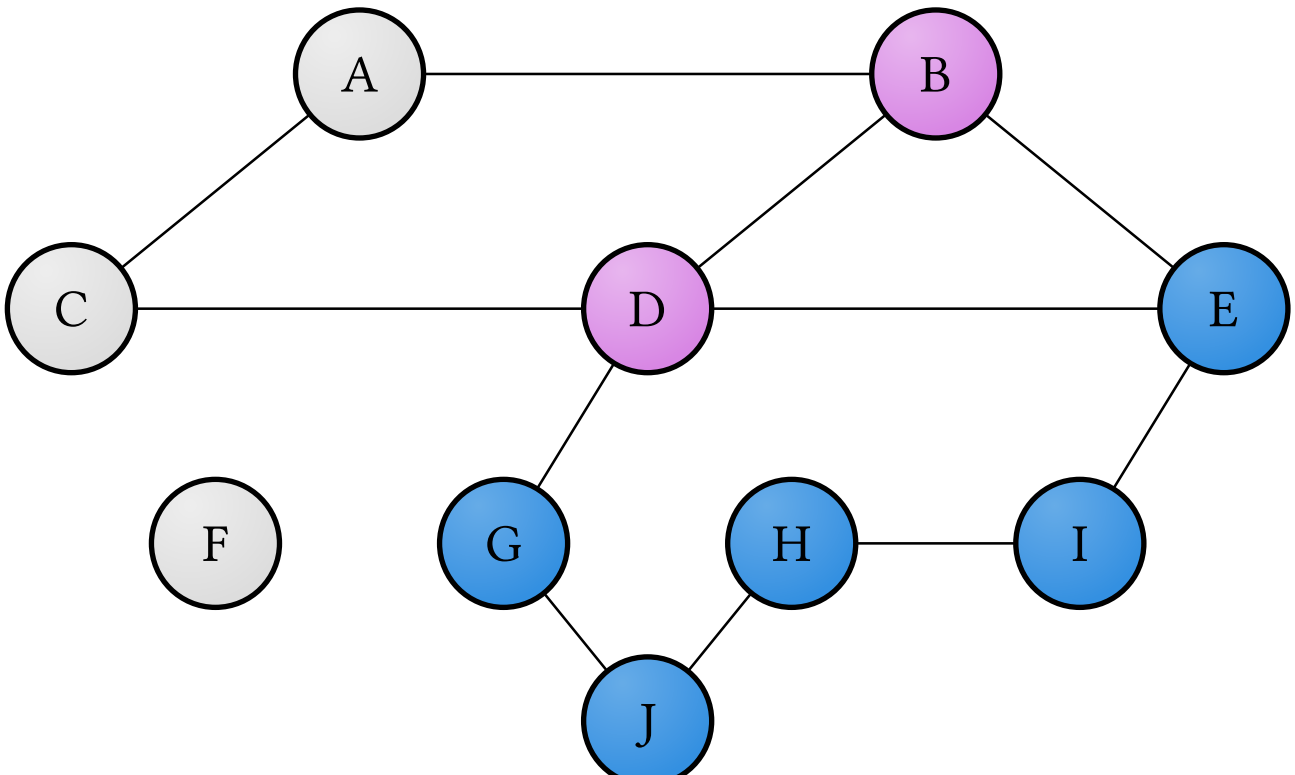
Zásobník:
[D, E]



Příklad DFS

Aktuální: B

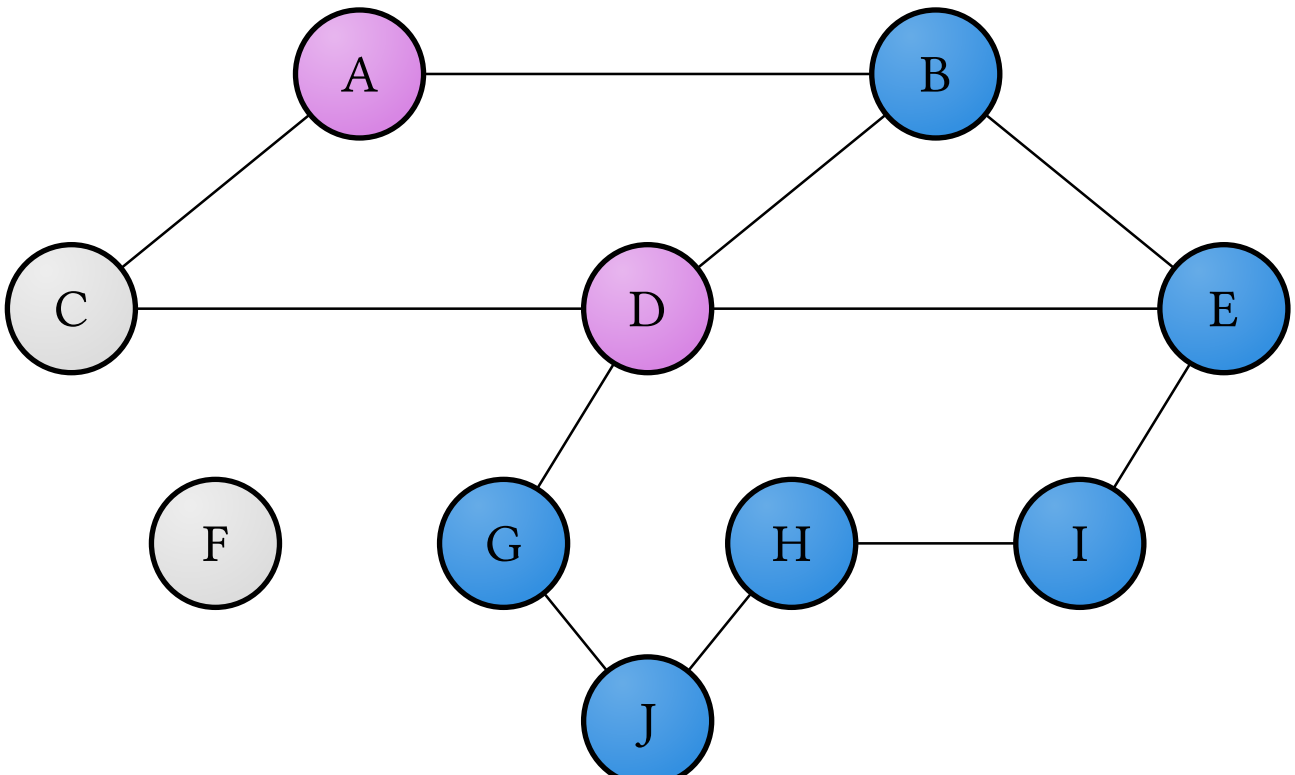
Zásobník:
[D, B]



Příklad DFS

Aktuální: A

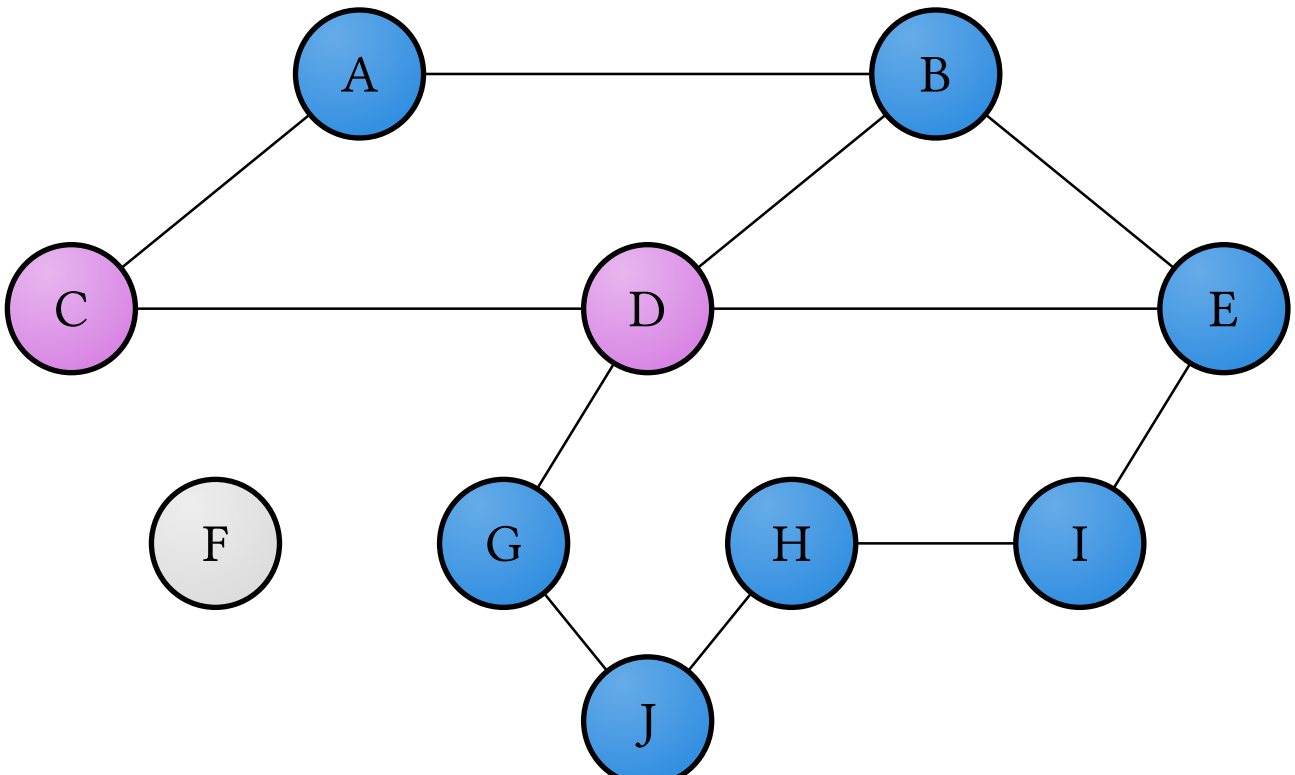
Zásobník:
[D, A]



Příklad DFS

Aktuální: C

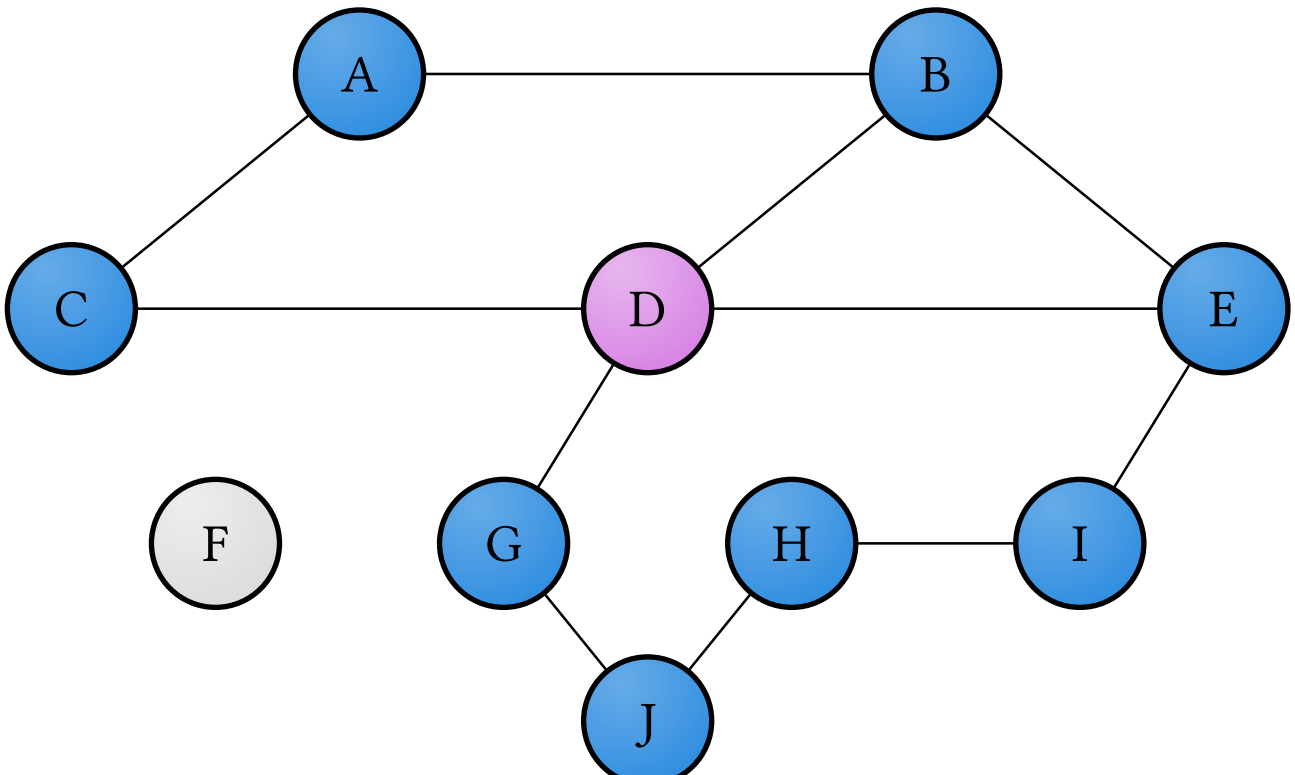
Zásobník:
[D, C]



Příklad DFS

Aktuální: D

Zásobník:
[D]



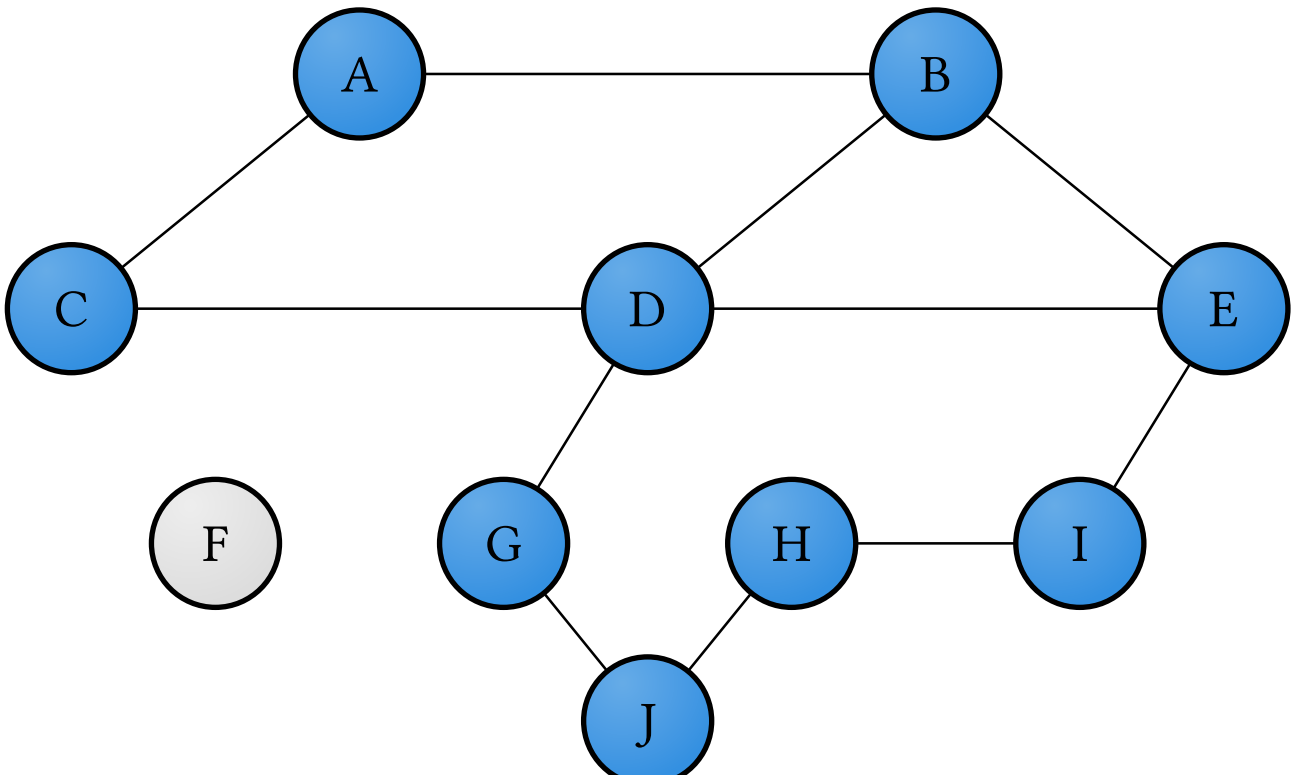
Příklad DFS

Aktuální: -

Zásobník:
[ ]

DFS dokončeno!

Vrchol F není dosažitelný z G.



Implementace DFS (rekurzivně)

```
function dfs(
  graph: Graph, vertex: string,
  visited: Set<string> = new Set()
): void {
  console.log(vertex); visited.add(vertex);
  for (const neighbor of graph[vertex]) {
    if (!visited.has(neighbor)) {
      dfs(graph, neighbor, visited);
    }
  }
}
```

Implementace DFS (iterativně)

```
function dfsIter(graph: Graph, start: string): void {
  const visited = new Set<string>();
  const stack = new Stack<string>();

  stack.push(start);

  while (!stack.isEmpty()) {
    const vertex = stack.pop()!;
```

Implementace DFS (iterativně)

```
    if (!visited.has(vertex)) {
      console.log(vertex); visited.add(vertex);

      for (const neighbor of graph[vertex]) {
        if (!visited.has(neighbor)) {
          stack.push(neighbor);
        }
      }
    }
  }
}
```

Porovnání BFS vs DFS

Porovnání BFS vs DFS

	BFS	DFS
Struktura	Fronta	Zásobník/rekurze
Složitost	$\mathcal{O}(\|V\| + \|E\|)$	$\mathcal{O}(\|V\| + \|E\|)$
Paměť	$\mathcal{O}(\|V\|)$	$\mathcal{O}(\|V\|)$
Nejkratší cesta	Ano	Ne
Detekce cyklů	Obtížnější	Jednodušší

Algoritmy na grafech

Praktické problémy řešené pomocí grafů:

Algoritmy na grafech

Praktické problémy řešené pomocí grafů:

- Jak najít **nejkratší cestu**?

Algoritmy na grafech

Praktické problémy řešené pomocí grafů:

- Jak najít **nejkratší cestu**?
- Jak propojit všechny vrcholy s **minimální cenou**?

Algoritmy na grafech

Praktické problémy řešené pomocí grafů:

- Jak najít **nejkratší cestu**?
- Jak propojit všechny vrcholy s **minimální cenou**?
- Obsahuje graf **cyklus**?

Algoritmy na grafech

Praktické problémy řešené pomocí grafů:

- Jak najít **nejkratší cestu**?
- Jak propojit všechny vrcholy s **minimální cenou**?
- Obsahuje graf **cyklus**?
- Jak najít **optimální pořadí** úloh?

Dijkstrův algoritmus

Princip:

Dijkstrův algoritmus

Princip:

- Hledá nejkratší cestu z jednoho vrcholu do všech ostatních
- Rozšíření BFS o váhy hran
- Používá prioritní frontu (min-heap)
- Greedy přístup - vždy vybere nejbližší nenavštívený vrchol

Dijkstrův algoritmus

Princip:

- Hledá nejkratší cestu z jednoho vrcholu do všech ostatních
- Rozšíření BFS o váhy hran
- Používá prioritní frontu (min-heap)
- Greedy přístup - vždy vybere nejbližší nenavštívený vrchol

Požadavky:

Dijkstrův algoritmus

Princip:

- Hledá nejkratší cestu z jednoho vrcholu do všech ostatních
- Rozšíření BFS o váhy hran
- Používá prioritní frontu (min-heap)
- Greedy přístup - vždy vybere nejbližší nenavštívený vrchol

Požadavky:

- Vážený graf bez záporných cyklů

Dijkstrův algoritmus

Princip:

- Hledá nejkratší cestu z jednoho vrcholu do všech ostatních
- Rozšíření BFS o váhy hran
- Používá prioritní frontu (min-heap)
- Greedy přístup - vždy vybere nejbližší nenavštívený vrchol

Požadavky:

- Vážený graf bez záporných cyklů

Složitost: $\mathcal{O}((|V| + |E|) \log |V|)$ s prioritní frontou

Bellman-Ford algoritmus

Princip:

Bellman-Ford algoritmus

Princip:

- Hledá nejkratší cestu z jednoho vrcholu do všech ostatních
- Opakuje **relaxaci** všech hran

Bellman-Ford algoritmus

Princip:

- Hledá nejkratší cestu z jednoho vrcholu do všech ostatních
- Opakuje **relaxaci** všech hran

Algoritmus:

Bellman-Ford algoritmus

Princip:

- Hledá nejkratší cestu z jednoho vrcholu do všech ostatních
- Opakuje **relaxaci** všech hran

Algoritmus:

- Projdi všechny hrany $|V| - 1$ krát
- V každé iteraci aktualizuj vzdálenosti
- Poslední iterace detekuje záporné cykly (pokud existují)

Bellman-Ford algoritmus

Princip:

- Hledá nejkratší cestu z jednoho vrcholu do všech ostatních
- Opakuje **relaxaci** všech hran

Algoritmus:

- Projdi všechny hrany $|V| - 1$ krát
- V každé iteraci aktualizuj vzdálenosti
- Poslední iterace detekuje záporné cykly (pokud existují)

Složitost: $\mathcal{O}(|V| \times |E|)$ - pomalejší než Dijkstra

Floyd-Warshall algoritmus

Princip:

Floyd-Warshall algoritmus

Princip:

- Najde vzdálenosti mezi **všemi páry** vrcholů najednou
- Postupně zvažuje cesty přes každý mezivrchol
- **Dynamické programování**

Floyd-Warshall algoritmus

Princip:

- Najde vzdálenosti mezi **všemi páry** vrcholů najednou
- Postupně zvažuje cesty přes každý mezivrchol
- **Dynamické programování**

Algoritmus:

Floyd-Warshall algoritmus

Princip:

- Najde vzdálenosti mezi **všemi páry** vrcholů najednou
- Postupně zvažuje cesty přes každý mezivrchol
- **Dynamické programování**

Algoritmus:

- Pro každý možný mezivrchol k
- Aktualizuj vzdálenost mezi každým párem (i, j)
- Použij vzorec: $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$

Floyd-Warshall algoritmus

Princip:

- Najde vzdálenosti mezi **všemi páry** vrcholů najednou
- Postupně zvažuje cesty přes každý mezivrchol
- **Dynamické programování**

Algoritmus:

- Pro každý možný mezivrchol k
- Aktualizuj vzdálenost mezi každým párem (i, j)
- Použij vzorec: $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$

Složitost: $\mathcal{O}(|V|^3)$ - použitelné pro malé grafy

Hashovací tabulka

Hashovací tabulka

Hashovací tabulka (Hash Table/Hash Map) je

Hashovací tabulka

Hashovací tabulka (Hash Table/Hash Map) je datová struktura pro

Hashovací tabulka

Hashovací tabulka (Hash Table/Hash Map) je datová struktura pro **rychlé vyhledávání**

Hashovací tabulka

Hashovací tabulka (Hash Table/Hash Map) je datová struktura pro **rychlé vyhledávání**

- Ukládá data jako

Hashovací tabulka

Hashovací tabulka (Hash Table/Hash Map) je datová struktura pro **rychlé vyhledávání**

- Ukládá data jako **páry klíč-hodnota**

Hashovací tabulka

Hashovací tabulka (Hash Table/Hash Map) je datová struktura pro **rychlé vyhledávání**

- Ukládá data jako **páry klíč-hodnota**
- Používá

Hashovací tabulka

Hashovací tabulka (Hash Table/Hash Map) je datová struktura pro **rychlé vyhledávání**

- Ukládá data jako **páry klíč-hodnota**
- Používá **hashovací funkci** pro výpočet indexu

Hashovací tabulka

Hashovací tabulka (Hash Table/Hash Map) je datová struktura pro **rychlé vyhledávání**

- Ukládá data jako **páry klíč-hodnota**
- Používá **hashovací funkci** pro výpočet indexu
- Poskytuje

Hashovací tabulka

Hashovací tabulka (Hash Table/Hash Map) je datová struktura pro **rychlé vyhledávání**

- Ukládá data jako **páry klíč-hodnota**
- Používá **hashovací funkci** pro výpočet indexu
- Poskytuje (amortizovanou) **konstantní** časovou složitost pro základní operace

Hashovací tabulka

Představte si telefonní seznam

Hashovací tabulka

Představte si telefonní seznam

Bez hashování:

- Lineární procházení všech kontaktů
- Hledání trvá dlouho

Hashovací tabulka

Představte si telefonní seznam

Bez hashování:

- Lineární procházení všech kontaktů
- Hledání trvá dlouho

S hashováním:

- Jméno $\longrightarrow$ hashovací funkce $\longrightarrow$ index
- Okamžitý přístup k číslu

Hashovací funkce

Hashovací funkce převádí

Hashovací funkce

Hashovací funkce převádí **klíč** na

Hashovací funkce

Hashovací funkce převádí **klíč** na **číselný index**

Hashovací funkce

Hashovací funkce převádí **klíč** na **číselný index**

Vlastnosti dobré hashovací funkce:

Hashovací funkce

Hashovací funkce převádí **klíč** na **číselný index**

Vlastnosti dobré hashovací funkce:

- **Deterministická** - stejný vstup = stejný výstup

Hashovací funkce

Hashovací funkce převádí **klíč** na **číselný index**

Vlastnosti dobré hashovací funkce:

- **Deterministická** - stejný vstup = stejný výstup
- **Uniformní distribuce** - rovnoměrné rozložení hodnot

Hashovací funkce

Hashovací funkce převádí **klíč** na **číselný index**

Vlastnosti dobré hashovací funkce:

- **Deterministická** - stejný vstup = stejný výstup
- **Uniformní distribuce** - rovnoměrné rozložení hodnot
- **Rychlá** - efektivní výpočet

Hashovací funkce

Hashovací funkce převádí **klíč** na **číselný index**

Vlastnosti dobré hashovací funkce:

- **Deterministická** - stejný vstup = stejný výstup
- **Uniformní distribuce** - rovnoměrné rozložení hodnot
- **Rychlá** - efektivní výpočet
- **Minimalizuje kolize** - různé klíče $\longrightarrow$ různé indexy

ASCII součet

```
function simpleHash(key: string, tableSize: number): number {
  let hash = 0;
  for (let i = 0; i < key.length; i++) {
    hash += key.charCodeAt(i);
  }
  return hash % tableSize;
}
```

ASCII součet

```
function simpleHash(key: string, tableSize: number): number {
  let hash = 0;
  for (let i = 0; i < key.length; i++) {
    hash += key.charCodeAt(i);
  }
  return hash % tableSize;
}
```

"abc" má ASCII součet =

ASCII součet

```
function simpleHash(key: string, tableSize: number): number {
  let hash = 0;
  for (let i = 0; i < key.length; i++) {
    hash += key.charCodeAt(i);
  }
  return hash % tableSize;
}
```

"abc" má ASCII součet = 97 + 98 + 99 = 294

ASCII součet

```
function simpleHash(key: string, tableSize: number): number {
  let hash = 0;
  for (let i = 0; i < key.length; i++) {
    hash += key.charCodeAt(i);
  }
  return hash % tableSize;
}
```

"abc" má ASCII součet = 97 + 98 + 99 = 294

Pro tabulku velikosti 10 ukládáme a hledáme hodnotu s klíčem "abc" na indexu

ASCII součet

```
function simpleHash(key: string, tableSize: number): number {
  let hash = 0;
  for (let i = 0; i < key.length; i++) {
    hash += key.charCodeAt(i);
  }
  return hash % tableSize;
}
```

`"abc"` má ASCII součet = 97 + 98 + 99 = 294

Pro tabulku velikosti 10 ukládáme a hledáme hodnotu s klíčem `"abc"` na indexu `294 % 10 = 4`

Kolize

Kolize nastává, když

Kolize

Kolize nastává, když dva různé klíče mají

Kolize

Kolize nastává, když dva různé klíče mají **stejný hash**

Kolize

Kolize nastává, když dva různé klíče mají **stejný hash**

Příklad:

Kolize

Kolize nastává, když dva různé klíče mají **stejný hash**

Příklad:

- `simpleHash("abc", 10) = 4`

Kolize

Kolize nastává, když dva různé klíče mají **stejný hash**

Příklad:

- `simpleHash("abc", 10) = 4`
- `simpleHash("cab", 10) = 4`

Kolize

Kolize nastává, když dva různé klíče mají **stejný hash**

Příklad:

- `simpleHash("abc", 10) = 4`
- `simpleHash("cab", 10) = 4`

Obě hodnoty bychom chtěli uložit na **stejnou pozici**

Kolize

Řešení kolizí:

Kolize

Řešení kolizí:

1. Zřetězení (Chaining)

Kolize

Řešení kolizí:

1. Zřetězení (Chaining)

- Každý index obsahuje (spojový) seznam prvků

Kolize

Řešení kolizí:

1. Zřetězení (Chaining)

- Každý index obsahuje (spojový) seznam prvků
- Více prvků se stejným hashem tvoří řetězec

Kolize

Řešení kolizí:

1. Zřetězení (Chaining)

- Každý index obsahuje (spojový) seznam prvků
- Více prvků se stejným hashem tvoří řetězec

2. Otevřené adresování (Open Addressing)

Kolize

Řešení kolizí:

1. Zřetězení (Chaining)

- Každý index obsahuje (spojový) seznam prvků
- Více prvků se stejným hashem tvoří řetězec

2. Otevřené adresování (Open Addressing)

- Při kolizi hledáme další volné místo

Kolize

Řešení kolizí:

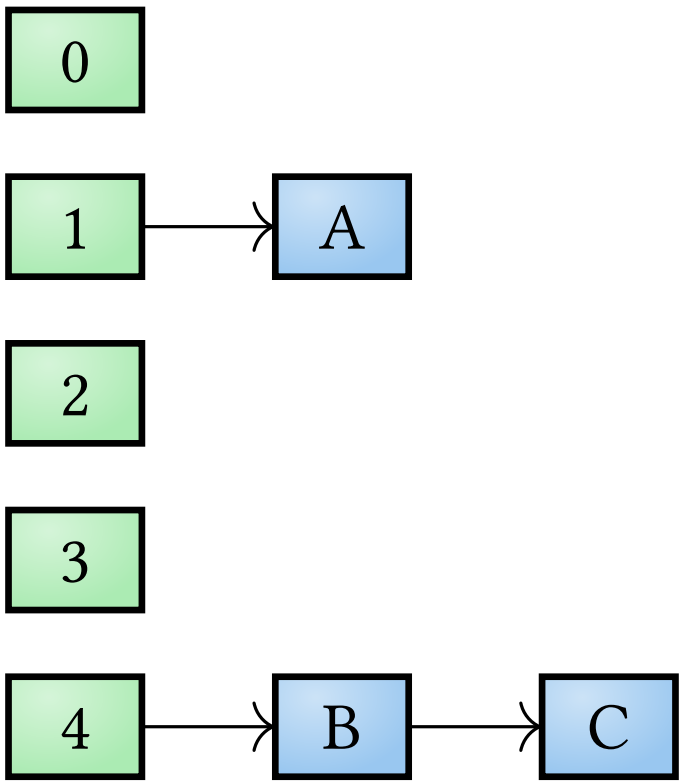
1. Zřetězení (Chaining)

- Každý index obsahuje (spojový) seznam prvků
- Více prvků se stejným hashem tvoří řetězec

2. Otevřené adresování (Open Addressing)

- Při kolizi hledáme další volné místo
- Lineární, kvadratické nebo dvojité hashování

Vizualizace zřetězení



Otevřené adresování

Otevřené adresování (Open Addressing) je

Otevřené adresování

Otevřené adresování (Open Addressing) je alternativní způsob řešení kolizí

Otevřené adresování

Otevřené adresování (Open Addressing) je alternativní způsob řešení kolizí

- Pokud je index i již obsazen,

Otevřené adresování

Otevřené adresování (Open Addressing) je alternativní způsob řešení kolizí

- Pokud je index i již obsazen, hledáme další volný index

Otevřené adresování

Otevřené adresování (Open Addressing) je alternativní způsob řešení kolizí

- Pokud je index i již obsazen, hledáme další volný index
- Všechny prvky jsou uloženy přímo v tabulce

Otevřené adresování

Otevřené adresování (Open Addressing) je alternativní způsob řešení kolizí

- Pokud je index i již obsazen, hledáme další volný index
- Všechny prvky jsou uloženy přímo v tabulce
- Provádíme tzv. **probing** (sondování)

Lineární probing

Princip

Lineární probing

Princip

Pokud je index i obsazen, zkusíme

Lineární probing

Princip

Pokud je index i obsazen, zkusíme $i + 1$, $i + 2$, $i + 3$, ...

Lineární probing

Princip

Pokud je index i obsazen, zkusíme $i + 1$, $i + 2$, $i + 3$, ...

Pokračujeme dokud

Lineární probing

Princip

Pokud je index i obsazen, zkusíme $i + 1$, $i + 2$, $i + 3$, ...

Pokračujeme dokud nenajdeme volné místo nebo

Lineární probing

Princip

Pokud je index i obsazen, zkusíme $i + 1$, $i + 2$, $i + 3$, ...

Pokračujeme dokud nenajdeme volné místo nebo neprojedeme celou tabulku

Lineární probing

Princip

Pokud je index i obsazen, zkusíme $i + 1$, $i + 2$, $i + 3$, ...

Pokračujeme dokud nenajdeme volné místo nebo neprojedeme celou tabulku (cyklicky se vracíme na začátek tabulky)

Lineární probing

Problém - clusterování

Lineární probing

Problém - clusterování

- Prvky se shlukují do souvislých bloků

Lineární probing

Problém - clusterování

- Prvky se shlukují do souvislých bloků
- Vyhledávání se zpomaluje

Lineární probing

Problém - clusterování

- Prvky se shlukují do souvislých bloků
- Vyhledávání se zpomaluje
- Čím větší cluster, tím větší pravděpodobnost dalšího růstu

Lineární probing

Příklad vkládání s lineárním probingem:

Lineární probing

Příklad vkládání s lineárním probingem:

```
// Table size 10, hash function: key % 10
hashTable.insert(23);  // h = 3, stored at index 3
hashTable.insert(13);  // h = 3, collision, store at index 4
hashTable.insert(33);  // h = 3, collision, store at index 5
hashTable.insert(43);  // h = 3, collision, store at index 6
```

Lineární probing

Příklad vkládání s lineárním probingem:

```
// Table size 10, hash function: key % 10
hashTable.insert(23);  // h = 3, stored at index 3
hashTable.insert(13);  // h = 3, collision, store at index 4
hashTable.insert(33);  // h = 3, collision, store at index 5
hashTable.insert(43);  // h = 3, collision, store at index 6
```

Vzniká cluster na indexech 3-6

Kvadratický probing

Princip

Kvadratický probing

Princip

Pokud je index i obsazen, zkusíme

Kvadratický probing

Princip

Pokud je index i obsazen, zkusíme $i + 1^2$, $i + 2^2$, $i + 3^2$, ...

Kvadratický probing

Princip

Pokud je index i obsazen, zkusíme $i + 1^2$, $i + 2^2$, $i + 3^2$, ...

Výhoda:

Kvadratický probing

Princip

Pokud je index i obsazen, zkusíme $i + 1^2$, $i + 2^2$, $i + 3^2$, ...

Výhoda: Menší clusterování než u lineárního probingu

Kvadratický probing

Princip

Pokud je index i obsazen, zkusíme $i + 1^2$, $i + 2^2$, $i + 3^2$, ...

Výhoda: Menší clusterování než u lineárního probingu

Nevýhoda:

Kvadratický probing

Princip

Pokud je index i obsazen, zkusíme $i + 1^2$, $i + 2^2$, $i + 3^2$, ...

Výhoda: Menší clusterování než u lineárního probingu

Nevýhoda: Může být obtížnější najít volné místo

Kvadratický probing

Příklad vkládání s kvadratickým probingem:

Kvadratický probing

Příklad vkládání s kvadratickým probingem:

```
// hash(key) = 3, table size 10
// Try indices:
(3 + 0**2) % 10  // = 3
(3 + 1**2) % 10  // = 4
(3 + 2**2) % 10  // = 7
(3 + 3**2) % 10  // = 12 % 10 = 2
(3 + 4**2) % 10  // = 19 % 10 = 9
// ...
```

Dvojité hashování

Princip

Dvojité hashování

Princip

Použijeme dvě různé hashovací funkce

Dvojité hashování

Princip

Použijeme dvě různé hashovací funkce

- První funkce $h_1(k)$ určí základní index

Dvojité hashování

Princip

Použijeme dvě různé hashovací funkce

- První funkce $h_1(k)$ určí základní index
- Při kolizi použijeme druhou funkci $h_2(k)$ pro výpočet velikosti kroku

Dvojité hashování

Princip

Použijeme dvě různé hashovací funkce

- První funkce $h_1(k)$ určí základní index
- Při kolizi použijeme druhou funkci $h_2(k)$ pro výpočet velikosti kroku
  - $h_1(k)$

Dvojité hashování

Princip

Použijeme dvě různé hashovací funkce

- První funkce $h_1(k)$ určí základní index
- Při kolizi použijeme druhou funkci $h_2(k)$ pro výpočet velikosti kroku
  - $h_1(k)$
  - $h_1(k) + h_2(k)$

Dvojité hashování

Princip

Použijeme dvě různé hashovací funkce

- První funkce $h_1(k)$ určí základní index
- Při kolizi použijeme druhou funkci $h_2(k)$ pro výpočet velikosti kroku
  - $h_1(k)$
  - $h_1(k) + h_2(k)$
  - $h_1(k) + 2h_2(k)$
  - ...

Dvojité hashování

Příklad dvojitého hashování:

Dvojité hashování

Příklad dvojitého hashování:

```
// Table size 10
const h1 = (key: number) => key % 10;       // Primary hash
const h2 = (key: number) => 7 - (key % 7); // Secondary hash
// For key 23:
h1(23); // = 3, try index 3
h2(23); // = 7 - (23 % 7) = 5, step is 5
// on first collision, try index 3 + 5 = 8
```

Dvojité hashování

Příklad dvojitého hashování:

```
// Table size 10
const h1 = (key: number) => key % 10;       // Primary hash
const h2 = (key: number) => 7 - (key % 7); // Secondary hash
// For key 23:
h1(23); // = 3, try index 3
h2(23); // = 7 - (23 % 7) = 5, step is 5
// on first collision, try index 3 + 5 = 8
```

Důležité: $h_2(k)$ nesmí být 0 a mělo by být nesoudělné s velikostí tabulky

Odstranění prvku

Problém při mazání:

Odstranění prvku

Problém při mazání:

Máme prvky na pozicích i_1, i_2, i_3 (díky kolizím)

Odstranění prvku

Problém při mazání:

Máme prvky na pozicích i_1, i_2, i_3 (díky kolizím)

Pokud odstraníme prvek na i_2:

Odstranění prvku

Problém při mazání:

Máme prvky na pozicích i_1, i_2, i_3 (díky kolizím)

Pokud odstraníme prvek na i_2:

- Při vyhledávání najdeme prázdné místo

Odstranění prvku

Problém při mazání:

Máme prvky na pozicích i_1, i_2, i_3 (díky kolizím)

Pokud odstraníme prvek na i_2:

- Při vyhledávání najdeme prázdné místo
- Myslíme si, že prvek neexistuje

Odstranění prvku

Problém při mazání:

Máme prvky na pozicích i_1, i_2, i_3 (díky kolizím)

Pokud odstraníme prvek na i_2:

- Při vyhledávání najdeme prázdné místo
- Myslíme si, že prvek neexistuje
- Ale prvek může být na i_3!

Odstranění prvku - náhrobek

Řešení - náhrobek (tombstone):

Odstranění prvku - náhrobek

Řešení - náhrobek (tombstone):

Místo smazání označíme pozici jako **smazanou**

Odstranění prvku - náhrobek

Řešení - náhrobek (tombstone):

Místo smazání označíme pozici jako **smazanou**

- Při vyhledávání:

Odstranění prvku - náhrobek

Řešení - náhrobek (tombstone):

Místo smazání označíme pozici jako **smazanou**

- Při vyhledávání: náhrobek ignorujeme a pokračujeme dál

Odstranění prvku - náhrobek

Řešení - náhrobek (tombstone):

Místo smazání označíme pozici jako **smazanou**

- Při vyhledávání: náhrobek ignorujeme a pokračujeme dál
- Při vkládání:

Odstranění prvku - náhrobek

Řešení - náhrobek (tombstone):

Místo smazání označíme pozici jako **smazanou**

- Při vyhledávání: náhrobek ignorujeme a pokračujeme dál
- Při vkládání: náhrobek můžeme přepsat novým prvkem

Shrnutí metod probingu

1. Lineární probing

- Hledáme na pozicích i, $i + 1$, $i + 2$, $i + 3$, ...

Shrnutí metod probingu

1. Lineární probing

- Hledáme na pozicích i, $i + 1$, $i + 2$, $i + 3$, ...

2. Kvadratický probing

- Hledáme na pozicích i, $i + 1^2$, $i + 2^2$, $i + 3^2$, ...

Shrnutí metod probingu

1. Lineární probing

- Hledáme na pozicích $i, i+1, i+2, i+3, \ldots$

2. Kvadratický probing

- Hledáme na pozicích $i, i+1^2, i+2^2, i+3^2, \ldots$

3. Dvojité hashování

- Hledáme na pozicích $i, i+f(k), i+2f(k), \ldots$
  kde $f(k)$ je jiná hashovací funkce

Časová složitost

Průměrný případ:

Časová složitost

Průměrný případ:

- Uložení prvku:

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$
- Vyhledání prvku:

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$
- Vyhledání prvku: $\mathcal{O}^*(1)$

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$
- Vyhledání prvku: $\mathcal{O}^*(1)$
- Odebrání prvku:

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$
- Vyhledání prvku: $\mathcal{O}^*(1)$
- Odebrání prvku: $\mathcal{O}^*(1)$

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$
- Vyhledání prvku: $\mathcal{O}^*(1)$
- Odebrání prvku: $\mathcal{O}^*(1)$

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$
- Vyhledání prvku: $\mathcal{O}^*(1)$
- Odebrání prvku: $\mathcal{O}^*(1)$

Nejhorší případ:

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$
- Vyhledání prvku: $\mathcal{O}^*(1)$
- Odebrání prvku: $\mathcal{O}^*(1)$

Nejhorší případ:

- Všechny operace:

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$
- Vyhledání prvku: $\mathcal{O}^*(1)$
- Odebrání prvku: $\mathcal{O}^*(1)$

Nejhorší případ:

- Všechny operace: $\mathcal{O}(n)$

Časová složitost

Průměrný případ:

- Uložení prvku: $\mathcal{O}^*(1)$
- Vyhledání prvku: $\mathcal{O}^*(1)$
- Odebrání prvku: $\mathcal{O}^*(1)$

Nejhorší případ:

- Všechny operace: $\mathcal{O}(n)$
  - Nastává při mnoha kolizích (všechny klíče v jednom bucketu)

Load Factor

Load Factor (faktor zaplnění)

$$\frac{\text{počet prvků}}{\text{velikost tabulky}}$$

Load Factor

Load Factor (faktor zaplnění)

$$\frac{\text{počet prvků}}{\text{velikost tabulky}}$$

- Určuje, jak

Load Factor

Load Factor (faktor zaplnění)

$$\frac{\text{počet prvků}}{\text{velikost tabulky}}$$

- Určuje, jak **zaplněná** je hashovací tabulka

Load Factor

Load Factor (faktor zaplnění)

$$\frac{\text{počet prvků}}{\text{velikost tabulky}}$$

- Určuje, jak **zaplněná** je hashovací tabulka
- Vyšší load factor =

Load Factor

Load Factor (faktor zaplnění)

$$\frac{\text{počet prvků}}{\text{velikost tabulky}}$$

- Určuje, jak **zaplněná** je hashovací tabulka
- Vyšší load factor = více kolizí

Load Factor

Load Factor (faktor zaplnění)

$$\frac{\text{počet prvků}}{\text{velikost tabulky}}$$

- Určuje, jak **zaplněná** je hashovací tabulka
- Vyšší load factor = více kolizí
- Optimální hodnota:

Load Factor

Load Factor (faktor zaplnění)

$$\frac{\text{počet prvků}}{\text{velikost tabulky}}$$

- Určuje, jak **zaplněná** je hashovací tabulka
- Vyšší load factor = více kolizí
- Optimální hodnota: ~ 0.7 (70% zaplnění)

Load Factor

Když load factor přesáhne limit:

Load Factor

Když load factor přesáhne limit:

- **Rehashing** - vytvoření větší tabulky

Load Factor

Když load factor přesáhne limit:

- **Rehashing** - vytvoření větší tabulky
- Všechny prvky se přesunou a přehashují

Výhody hashovací tabulky

Výhody hashovací tabulky

- **Velmi rychlé vyhledávání** - průměrně $\mathcal{O}^*(1)$

Výhody hashovací tabulky

- **Velmi rychlé vyhledávání** - průměrně $\mathcal{O}^*(1)$
- **Rychlé vkládání a mazání** - průměrně $\mathcal{O}^*(1)$

Výhody hashovací tabulky

- **Velmi rychlé vyhledávání** - průměrně $\mathcal{O}^*(1)$
- **Rychlé vkládání a mazání** - průměrně $\mathcal{O}^*(1)$
- **Flexibilní klíče** - lze použít jakýkoli typ

Výhody hashovací tabulky

- **Velmi rychlé vyhledávání** - průměrně $\mathcal{O}^*(1)$
- **Rychlé vkládání a mazání** - průměrně $\mathcal{O}^*(1)$
- **Flexibilní klíče** - lze použít jakýkoli typ
- **Ideální pro asociativní pole** - slovníky, cache, indexy

Nevýhody hashovací tabulky

Nevýhody hashovací tabulky

- **Negarantovaná rychlost** - nejhorší případ $\mathcal{O}(n)$

Nevýhody hashovací tabulky

- **Negarantovaná rychlost** - nejhorší případ $\mathcal{O}(n)$
- **Vyšší paměťová náročnost** - většinou není plně zaplněná

Nevýhody hashovací tabulky

- **Negarantovaná rychlost** - nejhorší případ $\mathcal{O}(n)$
- **Vyšší paměťová náročnost** - většinou není plně zaplněná
- **Neuspořádaná data** - nelze iterovat v pořadí vložení

Nevýhody hashovací tabulky

- **Negarantovaná rychlost** - nejhorší případ $\mathcal{O}(n)$
- **Vyšší paměťová náročnost** - většinou není plně zaplněná
- **Neuspořádaná data** - nelze iterovat v pořadí vložení
- **Kvalita závisí na hashovací funkci** - špatná funkce = pomalá tabulka

Hashovací tabulky v JavaScriptu

JavaScript (a tedy i TypeScript) mají vestavěné hashovací tabulky pomocí třídy

Hashovací tabulky v JavaScriptu

JavaScript (a tedy i TypeScript) mají vestavěné hashovací tabulky pomocí třídy `Map` a pomocí

Hashovací tabulky v JavaScriptu

JavaScript (a tedy i TypeScript) mají vestavěné hashovací tabulky pomocí třídy `Map` a pomocí objektů `Object`

Hashovací tabulky v JavaScriptu

```
const map = new Map<string, number>();
map.set("apple", 5);
map.set("banana", 3);
console.log(map.get("apple")); // 5
```

Hashovací tabulky v JavaScriptu

```
const map = new Map<string, number>();
map.set("apple", 5);
map.set("banana", 3);
console.log(map.get("apple")); // 5

const obj: Record<string, number> = {};
obj["apple"] = 5;
obj["banana"] = 3;
console.log(obj["apple"]); // 5
```

Hashovací tabulky v JavaScriptu

```
// Set - hash set (keys only)
const set = new Set<string>();
set.add("apple");
set.add("banana");
console.log(set.has("apple")); // true
```

Praktické využití

Kde se hashovací tabulky používají?

Praktické využití

Kde se hashovací tabulky používají?

- **Cache** - rychlé ukládání často používaných dat

Praktické využití

Kde se hashovací tabulky používají?

- **Cache** - rychlé ukládání často používaných dat
- **Databázové indexy** - rychlé vyhledávání záznamů

Praktické využití

Kde se hashovací tabulky používají?

- **Cache** - rychlé ukládání často používaných dat
- **Databázové indexy** - rychlé vyhledávání záznamů
- **Detekce duplicit** - kontrola existence prvků

Praktické využití

Kde se hashovací tabulky používají?

- **Cache** - rychlé ukládání často používaných dat
- **Databázové indexy** - rychlé vyhledávání záznamů
- **Detekce duplicit** - kontrola existence prvků
- **Počítání frekvencí** - histogram, statistiky

Praktické využití

Kde se hashovací tabulky používají?

- **Cache** - rychlé ukládání často používaných dat
- **Databázové indexy** - rychlé vyhledávání záznamů
- **Detekce duplicit** - kontrola existence prvků
- **Počítání frekvencí** - histogram, statistiky
- **Implementace Set** - množiny bez duplicit

Příklad

```
function wordCount(text: string): Map<string, number> {
  const counts = new Map<string, number>();
  const words = text.toLowerCase().split(/\s+/);
  for (const word of words) {
    counts.set(word, (counts.get(word) || 0) + 1);
  }
  return counts;
}
```

Příklad

```
function wordCount(text: string): Map<string, number> {
  const counts = new Map<string, number>();
  const words = text.toLowerCase().split(/\s+/);
  for (const word of words) {
    counts.set(word, (counts.get(word) || 0) + 1);
  }
  return counts;
}
```

Časová složitost: $\mathcal{O}(n)$ kde n je počet slov

Shrnutí

Klíčové poznatky

Co jsme se dnes naučili:

Klíčové poznatky

Co jsme se dnes naučili:

- **Graf** zobecňuje spojové seznamy i stromy

Klíčové poznatky

Co jsme se dnes naučili:

- **Graf** zobecňuje spojové seznamy i stromy
- Reprezentace: **matice sousednosti** vs. **seznam sousedů**

Klíčové poznatky

Co jsme se dnes naučili:

- **Graf** zobecňuje spojové seznamy i stromy
- Reprezentace: **matice sousednosti** vs. **seznam sousedů**
- **BFS** prochází do šířky (fronta), **DFS** do hloubky (zásobník)

Klíčové poznatky

Co jsme se dnes naučili:

- **Graf** zobecňuje spojové seznamy i stromy
- Reprezentace: **matice sousednosti** vs. **seznam sousedů**
- **BFS** prochází do šířky (fronta), **DFS** do hloubky (zásobník)
- **Hashovací tabulka** nabízí v průměru $\mathcal{O}(1)$ – za cenu kolizí

Klíčové poznatky

Co jsme se dnes naučili:

- **Graf** zobecňuje spojové seznamy i stromy
- Reprezentace: **matice sousednosti** vs. **seznam sousedů**
- **BFS** prochází do šířky (fronta), **DFS** do hloubky (zásobník)
- **Hashovací tabulka** nabízí v průměru $\mathcal{O}(1)$ – za cenu kolizí
- Kolize řešíme **zřetězením**, nebo **otevřeným adresováním**